Driver Drowsiness Detection System

Binary image classification (Drowsy vs Non-Drowsy) with MobileNetV2 transfer learning, face detection and TensorFlow Lite edge deployment

1. Business story & context

The problem. Driver fatigue is one of the leading causes of road accidents, especially in long-haul transport, where drivers spend many hours on monotonous roads, often at night. A drowsy driver reacts slowly, drifts out of lane and can fall asleep for several seconds. At highway speed a 3-second micro-sleep covers roughly 80 m of road with nobody in control.

Why manual monitoring and rule-based alerts fail.

- *Manual monitoring* (a co-driver or a dispatcher calling the driver) does not scale across a fleet and is not available at 3 a.m. Drivers also under-report their own tiredness.
- *Rule-based alerts* (hours-of-service limits, lane-departure warnings, steering-wheel variance) are indirect proxies. They fire only after driving has already degraded, and they cannot tell a tired driver from a rough road. A fixed rule such as "alert after 4 hours" ignores that fatigue depends on sleep debt, time of day and the individual.

How computer vision and deep learning help. Drowsiness is visible on the face: eyelids droop and close for longer, the mouth yawns and the head nods. A camera pointed at the driver, with a CNN classifying each frame, measures the *driver's state directly* instead of guessing from proxies. Because the model must run **inside the vehicle** (no reliable connectivity, no cloud latency), it has to be small and fast, which is why the project ends with TensorFlow Lite quantization.

What this notebook builds.

1. Inspect the Driver Drowsiness Dataset (cropped faces from the Real-Life Drowsiness Dataset videos).
2. Isolate the face with OpenCV (Viola-Jones/Haar) and Dlib (HOG + linear SVM).
3. Preprocess and augment, then train a MobileNetV2 classifier in two phases.
4. Evaluate with recall-focused metrics, choose a confidence threshold on a validation set and analyse errors.
5. Convert to TFLite (float32, dynamic-range, float16, int8) and benchmark on test images.
6. Answer the business questions with numbers from this notebook.

Convention. *Drowsy = 1 = positive class*, *Non-Drowsy = 0*. A **false negative** (a drowsy driver classified as alert) is the dangerous error.

2. Setup & reproducibility

All imports, one configuration cell with every hyperparameter, fixed random seeds and the library versions. The test set is touched only in the final evaluation sections.

```
import os, io, contextlib, random, re, time, hashlib
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"
from pathlib import Path

import numpy as np
import pandas as pd
import cv2
import dlib
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
                             classification_report, roc_curve, auc, precision_recall_curve, average_precision_score)
from IPython.display import display

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 100
```

```
# ---------------- Configuration: every tunable lives here ----------------
SEED = 42
SUBSET_FRACTION = 1.0        # <1.0 trains on a stratified fraction of the data (CPU speed-up). 1.0 = full data.
IMG_SIZE = 128               # uniform input size
BATCH_SIZE = 32
VAL_FRACTION = 0.15          # validation share carved out of train/

DATA_DIR = Path("dataset")
OUT_DIR = Path("outputs")
FIG_DIR = OUT_DIR / "figures"
CLASS_NAMES = ["non_drowsy", "drowsy"]   # list index == label; drowsy (1) is the positive class

FACE_PAD = 0.25              # reflect-padding added before Dlib (tight crops leave no margin for HOG)
```

```
FACE_MARGIN = 0.10           # extra context kept around the detected face box

AUG_ROTATION = 0.05          # fraction of a full turn  -> +/- 18 degrees
AUG_ZOOM = 0.15
AUG_BRIGHTNESS = 0.25

DROPOUT = 0.3
HEAD_UNITS = 128
P1_EPOCHS, P1_LR = 12, 1e-3                    # phase 1: frozen base, train the head
P2_EPOCHS, P2_LR, UNFREEZE_LAST = 12, 1e-5, 30   # phase 2: fine-tune last N base layers
ES_PATIENCE, LR_PATIENCE, LR_FACTOR = 4, 2, 0.3

MIN_PRECISION = 0.95         # "acceptable precision" floor on validation when choosing the alert threshold
SMOOTH_WINDOW = 10           # frames averaged before raising an alert (section 12)

for d in (OUT_DIR, FIG_DIR):
    d.mkdir(parents=True, exist_ok=True)

random.seed(SEED); np.random.seed(SEED); tf.keras.utils.set_random_seed(SEED)

print(f"Python libs  | tensorflow {tf.__version__} | keras {keras.__version__} | opencv {cv2.__version__} | "
      f"dlib {dlib.__version__} | scikit-learn {sklearn.__version__}")
print(f"             | numpy {np.__version__} | pandas {pd.__version__} | matplotlib {plt.matplotlib.__version__} | seaborn {sns.__version__}")
print("Devices      |", [d.device_type for d in tf.config.list_physical_devices()], "| CPU cores:", os.cpu_count())
```

```
Python libs  | tensorflow 2.16.2 | keras 3.3.3 | opencv 4.10.0 | dlib 20.0.1 | scikit-learn 1.7.2
             | numpy 1.26.4 | pandas 2.3.3 | matplotlib 3.10.8 | seaborn 0.13.2
Devices      | ['CPU'] | CPU cores: 16
```

```
def save_show(name):
    """Save the current figure into outputs/figures and display it."""
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"{name}.png", dpi=110, bbox_inches="tight")
    plt.show()


def show_images(images, titles, ncols, cell=(2.3, 2.5), suptitle=None):
    nrows = int(np.ceil(len(images) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(cell[0] * ncols, cell[1] * nrows), squeeze=False)
    for ax in axes.ravel():
        ax.axis("off")
    for ax, img, title in zip(axes.ravel(), images, titles):
        ax.imshow(img); ax.set_title(title, fontsize=8)
    if suptitle:
        fig.suptitle(suptitle, y=1.0)
```

3. Load & inspect data

We index the folder structure into a DataFrame (path, label, split). File names look like `A0022.png` or `ZB0037.png`: a letter prefix followed by a frame-like number. The prefix is upper-case for drowsy frames and lower-case for non-drowsy frames. We parse it (column `seq`) because it *might* identify a source video or subject, which matters for the leakage discussion later. The documents do **not** say what the prefix means, so we never assume it: the leakage checks below are also repeated with a purely pixel-based measure that does not depend on file names.

```
def build_index():
    rows = []
    for split in ("train", "test"):
        for label, cls in enumerate(CLASS_NAMES):
            for p in sorted((DATA_DIR / split / cls).glob("*")):
                if p.suffix.lower() in {".png", ".jpg", ".jpeg"}:
                    prefix, number = re.fullmatch(r"([A-Za-z]+)(\d+)", p.stem).groups()
                    rows.append((p, label, split, prefix, int(number)))
    return pd.DataFrame(rows, columns=["path", "label", "split", "seq", "frame"])


df = build_index()
if SUBSET_FRACTION < 1.0:
    df = (df.groupby(["split", "label"], group_keys=False)
            .apply(lambda g: g.sample(frac=SUBSET_FRACTION, random_state=SEED)).reset_index(drop=True))
    print(f"!! SUBSET MODE: using {SUBSET_FRACTION:.0%} of the data !!")
df["class_name"] = df.label.map(dict(enumerate(CLASS_NAMES)))
print(df.groupby(["split", "class_name"]).size().unstack())
print("\nExample rows:"); display(df.head(3))
```

```
class_name  drowsy  non_drowsy
split
test          1250        1250
train         3750        3750

Example rows:
```

	path	label	split	seq	frame	class_name
0	dataset\train\non_drowsy\a0006.png	0	train	a	6	non_drowsy
1	dataset\train\non_drowsy\a0009.png	0	train	a	9	non_drowsy
2	dataset\train\non_drowsy\a0013.png	0	train	a	13	non_drowsy

```
# Integrity checks: every file must decode to a 3-channel image; also hash files to find exact duplicates and label conflicts.
shapes, corrupted, hashes = [], [], []
for p in df.path:
    img = cv2.imread(str(p), cv2.IMREAD_COLOR)
```

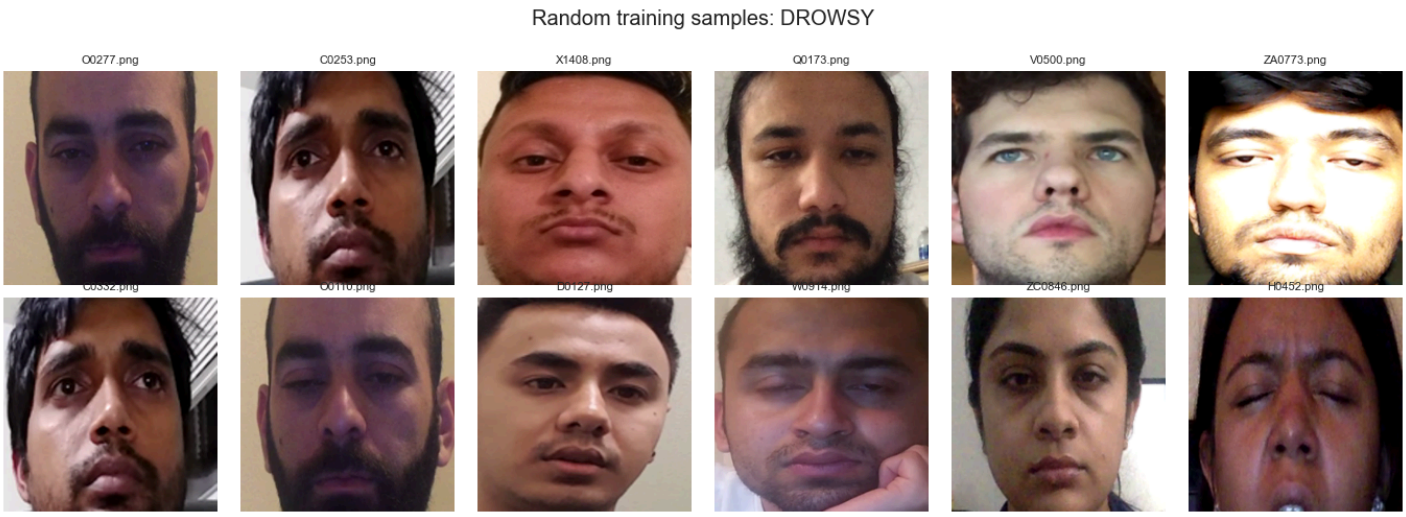
```
    if img is None:
        corrupted.append(p); shapes.append(None)
    else:
        shapes.append(img.shape)
    hashes.append(hashlib.md5(p.read_bytes()).hexdigest())
df["shape"] = shapes
df["md5"] = hashes

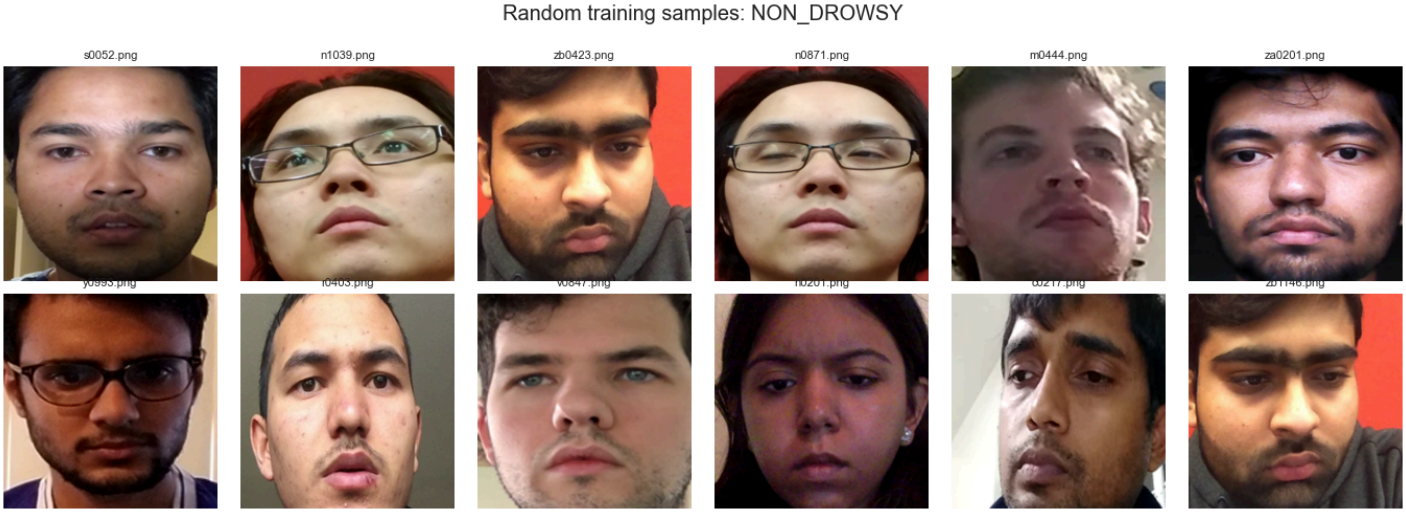
print("Corrupted / unreadable files :", len(corrupted))
print("Image shapes (H, W, C)       :", df["shape"].value_counts().to_dict())
dup = df[df.duplicated("md5", keep=False)]
conflict = df.groupby("md5").label.nunique().gt(1).sum()
print(f"Exact-duplicate files        : {len(dup)} files in {dup.md5.nunique()} groups")
print("Duplicate hashes with conflicting labels :", conflict)
cross = df.groupby("md5").split.nunique().gt(1).sum()
print("Duplicate hashes present in BOTH train and test :", cross)
```

```
Corrupted / unreadable files : 0
Image shapes (H, W, C)       : {(227, 227, 3): 10000}
Exact-duplicate files        : 0 files in 0 groups
Duplicate hashes with conflicting labels : 0
Duplicate hashes present in BOTH train and test : 0
```

```
rng = np.random.default_rng(SEED)
def sample_grid(split, label, n=12):
    sub = df[(df.split == split) & (df.label == label)]
    chosen = sub.iloc[rng.choice(len(sub), n, replace=False)]
    imgs = [cv2.cvtColor(cv2.imread(str(p)), cv2.COLOR_BGR2RGB) for p in chosen.path]   # OpenCV loads BGR -> convert for Matplotlib
    return imgs, [p.name for p in chosen.path]

for label in (1, 0):
    imgs, names = sample_grid("train", label)
    show_images(imgs, names, ncols=6, suptitle=f"Random training samples: {CLASS_NAMES[label].upper()}")
    save_show(f"samples_{CLASS_NAMES[label]}")
```

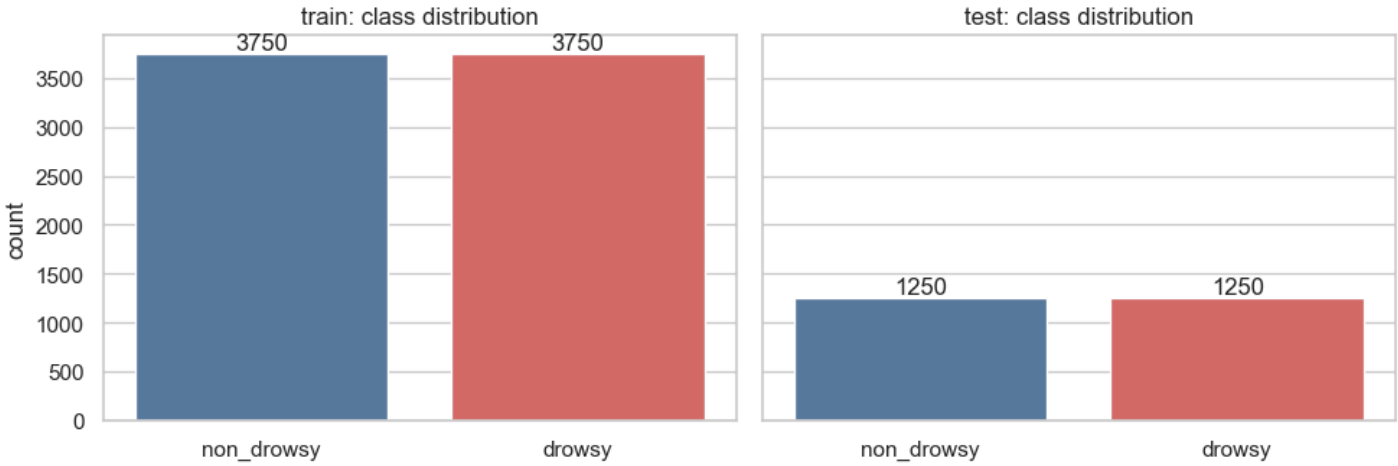





```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), sharey=True)
for ax, split in zip(axes, ("train", "test")):
    sns.countplot(data=df[df.split == split], x="class_name", hue="class_name", order=CLASS_NAMES, palette=["#4C78A8", "#E45756"], legend=False,
ax=ax)
    ax.set_title(f"{split}: class distribution"); ax.set_xlabel("")
    for c in ax.containers:
        ax.bar_label(c)
save_show("class_distribution")

# Brightness / contrast per class is a quick check for systematic acquisition differences that would let a model cheat.
stats = []
```

```
for p in df.sample(min(len(df), 1500), random_state=SEED).itertuples():
    g = cv2.imread(str(p.path), cv2.IMREAD_GRAYSCALE)
    stats.append((p.class_name, g.mean(), g.std()))
stats = pd.DataFrame(stats, columns=["class", "mean_brightness", "contrast"])
print(stats.groupby("class").agg(["mean", "std"]).round(1))
```



	mean_brightness		contrast	
	mean	std	mean	std
class				
drowsy	100.1	23.7	51.1	12.0
non_drowsy	109.9	18.7	55.7	10.7

What the inspection shows

- **Balance.** Train has 3,750 drowsy and 3,750 non-drowsy images; test has 1,250 of each. Both splits are exactly 50/50, so no class weighting is needed (the notebook confirms the balanced weights are 1.0 / 1.0 and does not use them).
- **Image quality.** All 10,000 files decode, all are 227x227 with 3 channels, and there are **0 corrupted files**, **0 exact duplicates** and **0 label conflicts** (no identical file appears under both labels, nor in both train and test).
- **Labelling.** A file-level check cannot detect wrong labels. Visually, the labels look plausible: closed or heavy-lidded eyes in the drowsy grid, open eyes in the non-drowsy grid. Section 10 shows that at least one non-drowsy frame (n0786) contains closed eyes (possibly a blink), so some label noise at the boundary is possible.
- **Possible shortcut.** Drowsy images are slightly darker on average (mean brightness 100.1 vs 109.9, contrast 51.1 vs 55.7 for non-drowsy). A model could partly learn "darker = drowsy". Section 10 tests how the model reacts to darkening.
- **Real-world caveat.** The dataset is perfectly balanced by construction. In real driving, drowsy frames are rare, which changes how accuracy and precision should be read (see Q7).

4. Face detection (OpenCV + Dlib)

Dlib's detector (`dlib.get_frontal_face_detector()`) is a *HOG + linear classifier*. The image is split into small cells and, per cell, a histogram of edge directions weighted by edge strength is computed (HOG). These histograms are normalised over blocks of neighbouring cells, which makes the descriptor fairly robust to lighting and contrast changes. A sliding window over an image pyramid scores each position with a linear SVM, and overlapping hits are merged into a bounding box. It is fast on CPU, needs no GPU, and works best on frontal or near-frontal faces.

OpenCV's Haar cascade is Viola-Jones: rectangle (Haar-like) intensity features evaluated with an integral image, chained in a cascade of boosted classifiers that quickly reject non-face windows. It is also what produced this dataset's crops, so it is a useful second opinion.

Why isolate the face? The camera sees the steering wheel, headrest, window, passengers and background. Without a face crop the CNN's limited pixel budget (128x128) is spent partly on irrelevant regions: the eyes, which carry the drowsiness signal, are smaller, and the model can latch onto *spurious cues* such as the background or the seat that happen to correlate with a class in this dataset but not in another vehicle.

Implementation note. The images are already tight face crops, so Dlib's HOG window has almost no context around the face. We reflect-pad the image by 25% before detection (and map the box back), which makes detection more reliable. If no face is found we **fall back to the original image**, so no sample is dropped.

Does it pay off here? The no-crop ablation in section 12 tests this directly (result: no measurable gain on this dataset).

```
dlib_detector = dlib.get_frontal_face_detector()
haar_detector = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")


def detect_dlib(rgb):
    """Largest Dlib HOG face box (x1, y1, x2, y2) in original coordinates, or None."""
    h, w = rgb.shape[:2]
    pad = int(FACE_PAD * h)
    padded = cv2.copyMakeBorder(rgb, pad, pad, pad, pad, cv2.BORDER_REFLECT)
    rects = dlib_detector(padded, 0)
    if not rects:
```

```
        return None
    r = max(rects, key=lambda r: r.width() * r.height())
    return (max(r.left() - pad, 0), max(r.top() - pad, 0), min(r.right() - pad, w), min(r.bottom() - pad, h))


def detect_haar(rgb):
    """Largest Viola-Jones (Haar cascade) face box (x1, y1, x2, y2), or None."""
    boxes = haar_detector.detectMultiScale(cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY), scaleFactor=1.1, minNeighbors=5)
    if len(boxes) == 0:
        return None
    x, y, w, h = max(boxes, key=lambda b: b[2] * b[3])
    return (x, y, x + w, y + h)

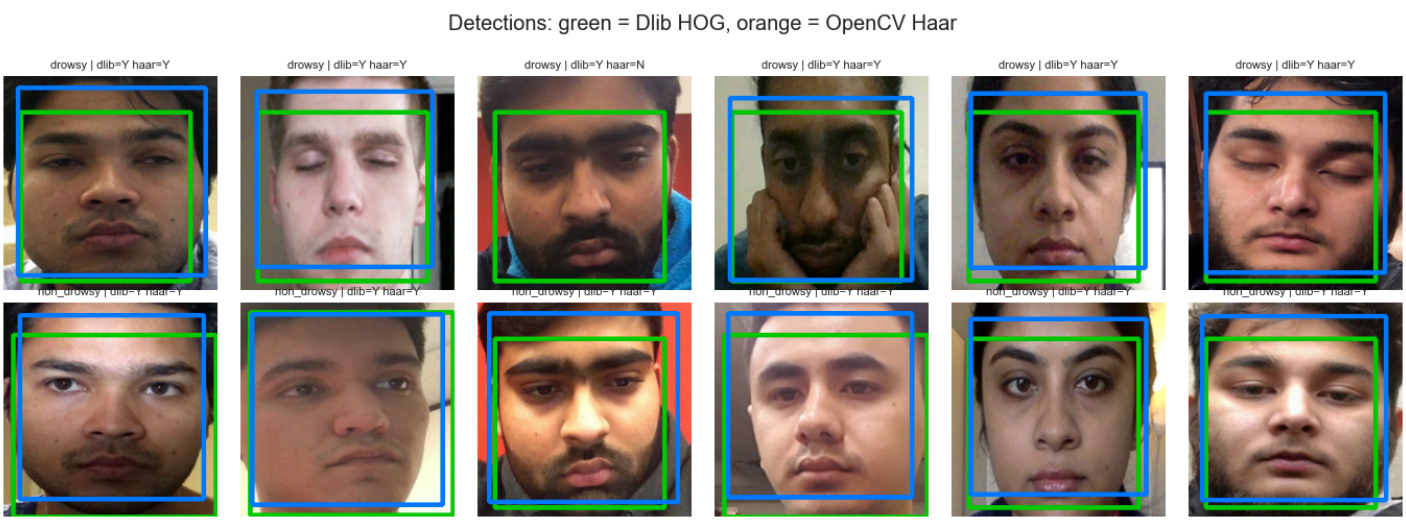

def crop_to_box(rgb, box):
    """Crop with a small margin; fall back to the full image when no face was found."""
    if box is None:
        return rgb
    h, w = rgb.shape[:2]
    x1, y1, x2, y2 = box
    mx, my = int(FACE_MARGIN * (x2 - x1)), int(FACE_MARGIN * (y2 - y1))
    return rgb[max(y1 - my, 0):min(y2 + my, h), max(x1 - mx, 0):min(x2 + mx, w)]


def load_rgb(path):
    return cv2.cvtColor(cv2.imread(str(path)), cv2.COLOR_BGR2RGB)   # BGR -> RGB


def annotate(rgb, dbox, hbox):
    out = rgb.copy()
    for box, colour in ((dbox, (0, 200, 0)), (hbox, (0, 120, 255))):    # green = Dlib, orange = Haar
        if box is not None:
            cv2.rectangle(out, box[:2], box[2:], colour, 3)
    return out


demo_rows = []
for label in (1, 0):
    sub = df[(df.split == "train") & (df.label == label)].sample(6, random_state=SEED)
    demo_rows += [load_rgb(p) for p in sub.path]
detections = [(detect_dlib(im), detect_haar(im)) for im in demo_rows]
show_images([annotate(im, d, h) for im, (d, h) in zip(demo_rows, detections)],
            [f"{'drowsy' if i < 6 else 'non_drowsy'} | dlib={'Y' if d else 'N'} haar={'Y' if h else 'N'}"
             for i, (d, h) in enumerate(detections)], ncols=6, suptitle="Detections: green = Dlib HOG, orange = OpenCV Haar")
save_show("face_detections")

show_images([im for im in demo_rows[:6]] + [crop_to_box(im, d) for im, (d, _) in zip(demo_rows[:6], detections[:6])],
            ["before"] * 6 + ["after (Dlib crop)"] * 6, ncols=6, suptitle="Before / after face crop (drowsy samples)")
save_show("face_crops_before_after")
```





Running the detectors on every image

Each image is loaded as RGB, both detectors are run (for the detection-rate comparison), the **Dlib crop** (with fallback) is resized to 128x128 with area interpolation and stored as `uint8`. The pixel values are converted to the 0-1 range later, inside the `tf.data` pipeline.

```
def preprocess_image(path):
    rgb = load_rgb(path)
    dbox, hbox = detect_dlib(rgb), detect_haar(rgb)
    face = cv2.resize(crop_to_box(rgb, dbox), (IMG_SIZE, IMG_SIZE), interpolation=cv2.INTER_AREA)
    area_kept = 1.0 if dbox is None else (dbox[2] - dbox[0]) * (dbox[3] - dbox[1]) / (rgb.shape[0] * rgb.shape[1])
    return face, dbox is not None, hbox is not None, area_kept


t0 = time.time()
results = [preprocess_image(p) for p in df.path]
X_all = np.stack([r[0] for r in results])                # (N, 128, 128, 3) uint8
df["dlib_found"] = [r[1] for r in results]
df["haar_found"] = [r[2] for r in results]
df["face_area_frac"] = [r[3] for r in results]
y_all = df.label.to_numpy()
print(f"Processed {len(df)} images in {time.time() - t0:.0f}s -> array {X_all.shape}, dtype {X_all.dtype}")

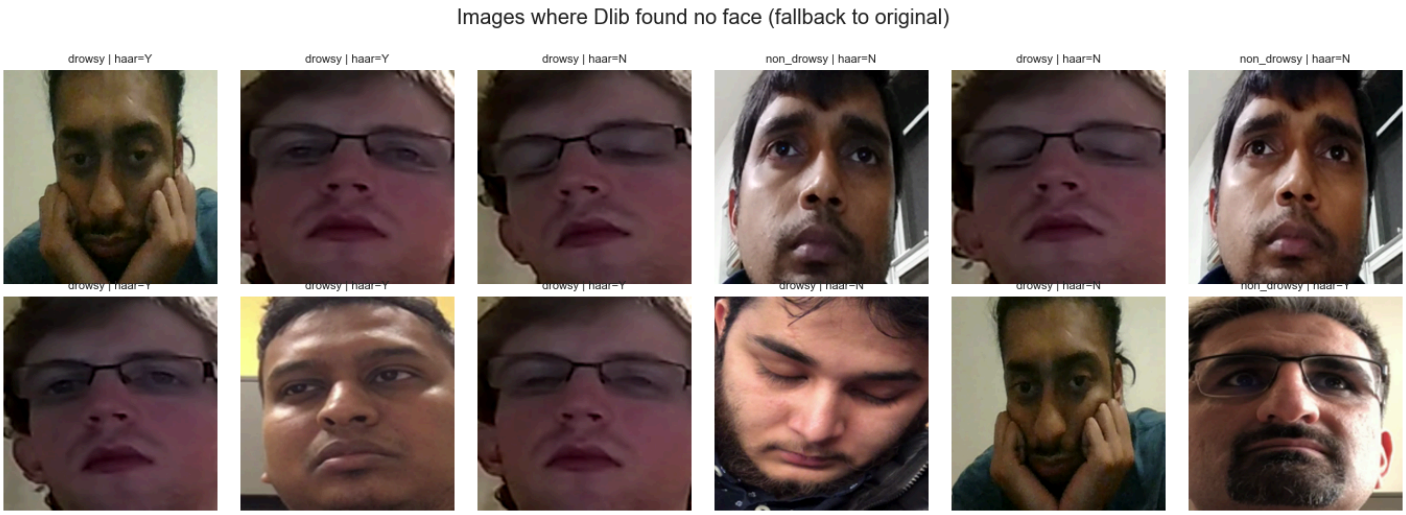
rate = (df.groupby(["split", "class_name"])[["dlib_found", "haar_found"]].mean() * 100).round(1)
rate.columns = ["Dlib detection rate %", "Haar detection rate %"]
display(rate)
print("Overall Dlib detection rate: %.1f%% | Haar: %.1f%% | at least one: %.1f%%" % (
    df.dlib_found.mean() * 100, df.haar_found.mean() * 100, (df.dlib_found | df.haar_found).mean() * 100))
print("Median share of the image area covered by the Dlib box (found only): %.2f" % df.loc[df.dlib_found, "face_area_frac"].median())
```

```
Processed 10000 images in 322s -> array (10000, 128, 128, 3), dtype uint8
```

split	class_name	Dlib detection rate %	Haar detection rate %
test	drowsy	93.3	90.5
	non_drowsy	96.4	91.8
train	drowsy	93.1	91.8
	non_drowsy	96.9	91.4

```
Overall Dlib detection rate: 95.0% | Haar: 91.5% | at least one: 98.2%
Median share of the image area covered by the Dlib box (found only): 0.63
```

```
# What do the fallback cases look like? (no Dlib face -> the original image is used)
missed = df[~df.dlib_found].sample(12, random_state=SEED)
show_images([load_rgb(p) for p in missed.path], [f"{c} | haar={'Y' if h else 'N'}" for c, h in zip(missed.class_name, missed.haar_found)],
            ncols=6, suptitle="Images where Dlib found no face (fallback to original)")
save_show("face_fallbacks")
```



What the detectors show

- **Dlib (HOG)** found a face in **95.0%** of all images (train drowsy 93.1%, train non-drowsy 96.9%, test drowsy 93.3%, test non-drowsy 96.4%). **OpenCV Haar** found a face in **91.5%**. At least one of them succeeded for **98.2%** of the images.
- Detection is **lower for drowsy faces** (about 93% vs 96-97%) with both detectors. This fits the data: closed eyes, lowered or tilted heads and off-frontal poses are exactly the situations where frontal-face detectors struggle.
- Where a face was found, the Dlib box covers a **median of 63%** of the 227x227 image. So roughly a third of the pixels in the supplied "cropped" images are outside the face box (hair, shoulders, background such as the red wall or blue clothing visible in the samples). The crop removes that context.
- The fallback grid shows what the remaining ~5% look like: hands resting on the face, heads tilted down or back, glasses, and faces that fill the whole frame so there is no margin for the detector. They are kept at full size, so no sample is lost.
- The processing step (loading, both detectors and resizing for 10,000 images) took about **320 s** on CPU.

5. Preprocessing & augmentation

5.1 Train / validation / test split

The provided `test/` folder is kept **untouched** until section 8. A stratified validation set is carved from `train/` and used for early stopping, learning-rate scheduling, checkpointing and threshold selection.

> **Caveat (checked below).** A random split of video *frames* puts near-identical neighbouring frames on both sides of the split. Without a subject ID this cannot be fully prevented, so we measure it and report it as a limitation.

```
is_train_pool = (df.split == "train").to_numpy()
pool_idx = np.flatnonzero(is_train_pool)
tr_idx, va_idx = train_test_split(pool_idx, test_size=VAL_FRACTION, stratify=y_all[pool_idx], random_state=SEED)
te_idx = np.flatnonzero(~is_train_pool)

X_train, y_train = X_all[tr_idx], y_all[tr_idx]
X_val, y_val = X_all[va_idx], y_all[va_idx]
X_test, y_test = X_all[te_idx], y_all[te_idx]
df_train, df_val, df_test = (df.iloc[i].reset_index(drop=True) for i in (tr_idx, va_idx, te_idx))

for name, y in (("train", y_train), ("val", y_val), ("test", y_test)):
    print(f"{name:5s}: {len(y):5d} images | drowsy {y.mean():.1%}")

# Class weights: only applied if the training set is actually imbalanced.
weights = compute_class_weight("balanced", classes=np.array([0, 1]), y=y_train)
imbalanced = max(weights) / min(weights) > 1.2
class_weight = dict(enumerate(weights)) if imbalanced else None
print("Balanced class weights:", dict(enumerate(weights.round(3))), "-> used in training:", imbalanced)
```

```
train:  6375 images | drowsy 50.0%
val  :  1125 images | drowsy 50.0%
test :  2500 images | drowsy 50.0%
Balanced class weights: {0: 1.0, 1: 1.0} -> used in training: False
```

5.2 Near-duplicate leakage check

Two independent checks on how similar validation and test frames are to training frames:

1. **File-name adjacency.** For each image, the frame-number distance to the closest training frame with the same file-name letter (assumed here to identify the source sequence).
2. **Pixel similarity.** Distance (on 32x32 grayscale face crops) to the nearest training image, compared with the distance to a random training image.

```python
def nearest_frame_gap(query, ref):
    gaps = []
    for seq, g in query.groupby("seq"):
        frames = np.sort(ref.loc[ref.seq == seq, "frame"].to_numpy())
        if len(frames) == 0:
            gaps += [np.nan] * len(g); continue
        pos = np.searchsorted(frames, g.frame.to_numpy())
        left = np.abs(g.frame.to_numpy() - frames[np.clip(pos - 1, 0, len(frames) - 1)])
        right = np.abs(g.frame.to_numpy() - frames[np.clip(pos, 0, len(frames) - 1)])
        gaps += list(np.minimum(left, right))
    return np.array(gaps)

for name, q in (("validation", df_val), ("test", df_test)):
    gaps = nearest_frame_gap(q, df_train)
    print(f"{name:10s} -> nearest train frame with same letter: median gap {np.nanmedian(gaps):.0f} frames, "
          f"{np.mean(gaps <= 5):.0%} within 5 frames, {np.mean(gaps <= 20):.0%} within 20 frames, "
          f"{np.mean(np.isnan(gaps)):.0%} have no train frame with that letter")
print("Letters shared by train and test:", len(set(df_train.seq) & set(df_test.seq)), "of", df_test.seq.nunique(), "test letters")
```

```
validation -> nearest train frame with same letter: median gap 3 frames, 78% within 5 frames, 100% within 20 frames, 0% have no train frame with that
letter
test       -> nearest train frame with same letter: median gap 3 frames, 78% within 5 frames, 100% within 20 frames, 0% have no train frame with that
letter
Letters shared by train and test: 54 of 54 test letters
```

```python
def small_features(X):
    return np.stack([cv2.resize(cv2.cvtColor(x, cv2.COLOR_RGB2GRAY), (32, 32), interpolation=cv2.INTER_AREA) for x in X]).reshape(len(X), -1) / 255.0

f_train, f_val, f_test = small_features(X_train), small_features(X_val), small_features(X_test)
nn = NearestNeighbors(n_neighbors=1).fit(f_train)
d_val, i_val = nn.kneighbors(f_val)
d_test, i_test = nn.kneighbors(f_test)
d_random = np.linalg.norm(f_val - f_train[rng.permutation(len(f_train))[:len(f_val)]], axis=1)
print(f"Median L2 distance | val -> nearest train: {np.median(d_val):.2f} | test -> nearest train: {np.median(d_test):.2f} "
      f"| val -> random train image: {np.median(d_random):.2f}")

pick = rng.choice(len(X_val), 5, replace=False)
imgs, titles = [], []
for k in pick:
    imgs += [X_val[k], X_train[i_val[k, 0]]]
    titles += [f"val ({CLASS_NAMES[y_val[k]]})", f"nearest train (d={d_val[k, 0]:.2f})"]
show_images(imgs, titles, ncols=10, cell=(1.8, 2.0), suptitle="Validation images and their nearest training image")
save_show("leakage_nearest_neighbours")
```

```
Median L2 distance | val -> nearest train: 0.86 | test -> nearest train: 0.87 | val -> random train image: 8.68
```



Leakage finding (limitation, quantified)

No subject ID is provided, so a subject-disjoint split is impossible. The file names are the only hint, and the documents do not say what the prefix means, so the findings below do not rely on it.

Measure	Validation	Test
Median pixel distance to the nearest training image (32x32 grayscale, L2)	**0.86**	**0.87**
... to a random training image	8.68	8.68
Median frame-number gap to the closest training frame with the same prefix	3	3
Share within 5 / 20 frames	78% / 100%	78% / 100%
Test prefixes that also occur in train		54 of 54

- **Pixel evidence (independent of the file names).** The nearest training image is about **10x closer** than a random one, and the side-by-side figure shows pairs that look like consecutive frames of the same person.
- **File-name evidence (assumption-dependent).** If the prefix identifies a source video or person, the data are interleaved frames: every test prefix occurs in train and nearly all images have a training frame within a few frame numbers.
- **The test set is as close to the training set as the validation set is.** The official test split is therefore **not** an independent check on unseen drivers; it measures frame-level generalisation within sequences the model has already seen. Treat all scores in this notebook as an optimistic upper bound for deployment on new drivers.

5.3 Augmentation

Applied to the **training set only**, on the fly, with Keras preprocessing layers inside the `tf.data` pipeline: horizontal flip (left/right head orientation), brightness (cabin lighting), zoom (camera distance) and rotation (head tilt). Rotation and zoom use reflected borders so no black corners appear. Pixels are first scaled to 0-1.

```python
augment = keras.Sequential([
    layers.RandomFlip("horizontal", seed=SEED),
    layers.RandomRotation(AUG_ROTATION, fill_mode="reflect", seed=SEED),
    layers.RandomZoom((-AUG_ZOOM, AUG_ZOOM), fill_mode="reflect", seed=SEED),
```

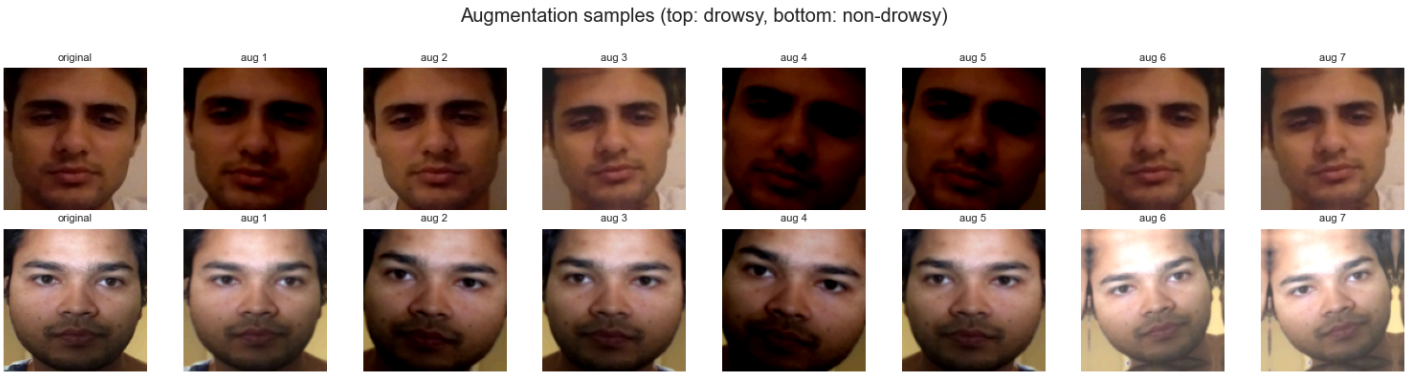
```
    layers.RandomBrightness(AUG_BRIGHTNESS, value_range=(0.0, 1.0), seed=SEED),
], name="augmentation")

AUTOTUNE = tf.data.AUTOTUNE

def make_dataset(X, y, training):
    """uint8 arrays -> batched float32 [0,1] tensors; shuffled + augmented when training."""
    ds = tf.data.Dataset.from_tensor_slices((X, y.astype("float32")))
    if training:
        ds = ds.shuffle(len(X), seed=SEED)
    ds = ds.batch(BATCH_SIZE).map(lambda a, b: (tf.cast(a, tf.float32) / 255.0, b), num_parallel_calls=AUTOTUNE)
    if training:
        ds = ds.map(lambda a, b: (tf.clip_by_value(augment(a, training=True), 0.0, 1.0), b), num_parallel_calls=AUTOTUNE)
    return ds.prefetch(AUTOTUNE)

# Visualise: the same drowsy and non-drowsy face, augmented 7 times each
rows = []
for label in (1, 0):
    img = X_train[np.flatnonzero(y_train == label)[0]].astype("float32") / 255.0
    batch = augment(np.repeat(img[None], 7, axis=0), training=True).numpy().clip(0, 1)
    rows += [img] + list(batch)
show_images(rows, ["original"] + [f"aug {i}" for i in range(1, 8)] + ["original"] + [f"aug {i}" for i in range(1, 8)],
            ncols=8, cell=(1.9, 2.0), suptitle="Augmentation samples (top: drowsy, bottom: non-drowsy)")
save_show("augmentation_samples")

train_ds = make_dataset(X_train, y_train, training=True)
val_ds = make_dataset(X_val, y_val, training=False)
test_ds = make_dataset(X_test, y_test, training=False)
xb, yb = next(iter(train_ds))
print(f"Training batch: {xb.shape}, dtype {xb.dtype}, pixel range [{xb.numpy().min():.2f}, {xb.numpy().max():.2f}], labels {yb.shape}")
```




```
Training batch: (32, 128, 128, 3), dtype <dtype: 'float32'>, pixel range [0.00, 1.00], labels (32,)
```

Pipeline check

- The training batch has shape (32, 128, 128, 3), float32, in the range [0, 1]. Images were converted BGR -> RGB at load time, face-cropped and resized to 128x128 once, then scaled to 0-1 per batch.
- The augmentation samples show the intended effects: mirrored faces, tilts of up to about 18 degrees, zoom in/out by up to 15% and brightness changes of up to 25%. Reflected borders avoid black corners that the model could use as a cue.
- Augmentation is applied **only to the training dataset**; validation and test batches are only scaled.
- **Limit of the augmentation.** Brightness changes are mild (about +/-25%). The model never sees genuinely dark cabins, which is quantified in the stress test in section 10.
- `tf.data` keeps the uint8 arrays in memory, shuffles, batches, augments in parallel (`AUTOTUNE`) and prefetches, so the CPU is not idle waiting for data.

6. Model: MobileNetV2 + custom head

Architecture. Input (128x128x3, values in 0-1) -> `Rescaling` to [-1, 1] (what MobileNetV2's ImageNet weights expect) -> **MobileNetV2** (ImageNet weights, `include_top=False`) -> global average pooling -> dropout -> dense(128, ReLU) -> dropout -> dense(1, sigmoid). The sigmoid output is the probability of *Drowsy*.

Why MobileNetV2?

- It uses *depthwise-separable convolutions* and *inverted residual blocks*, which give a small parameter count (about 2.2 M in the backbone) and low compute, designed for mobile and embedded hardware.
- It quantizes well, and TFLite supports all of its operators natively.
- ImageNet features (edges, textures, eye- and face-like patterns) transfer well to faces, which matters because the dataset is small by deep-learning standards.
- Alternatives are heavier (ResNet50 about 25 M parameters, VGG16 about 138 M) or less deployment-friendly; for a CPU-only in-vehicle camera MobileNetV2 is the best size/accuracy trade-off.

The base model is always called with `training=False` , so its BatchNorm layers keep their ImageNet statistics, also during fine-tuning.

```
def build_model():
    base = keras.applications.MobileNetV2(input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False, weights="imagenet")
    base.trainable = False
```

```
    inputs = keras.Input((IMG_SIZE, IMG_SIZE, 3), name="face_0_1")
    x = layers.Rescaling(2.0, offset=-1.0, name="to_minus1_1")(inputs)
    x = base(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(DROPOUT)(x)
    x = layers.Dense(HEAD_UNITS, activation="relu")(x)
    x = layers.Dropout(DROPOUT)(x)
    outputs = layers.Dense(1, activation="sigmoid", name="p_drowsy")(x)
    return keras.Model(inputs, outputs, name="drowsiness_mobilenetv2"), base


def compile_model(model, lr):
    model.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy",
                  metrics=["accuracy", keras.metrics.Precision(name="precision"), keras.metrics.Recall(name="recall"),
                           keras.metrics.AUC(name="auc")])


model, base = build_model()
compile_model(model, P1_LR)
model.summary()
print(f"\nMobileNetV2 backbone: {len(base.layers)} layers, {base.count_params():,} parameters (all frozen in phase 1)")
```

Model: "drowsiness_mobilenetv2"

Layer (type)	Output Shape	Param #
face_0_1 (InputLayer)	(None, 128, 128, 3)	0
to_minus1_1 (Rescaling)	(None, 128, 128, 3)	0
mobilenetv2_1.00_128 (Functional)	(None, 4, 4, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
p_drowsy (Dense)	(None, 1)	129

Total params: 2,422,081 (9.24 MB)

Trainable params: 164,097 (641.00 KB)

Non-trainable params: 2,257,984 (8.61 MB)

```
MobileNetV2 backbone: 154 layers, 2,257,984 parameters (all frozen in phase 1)
```

6.1 Baseline: small CNN trained from scratch

To make the "why not start from scratch?" question (business question Q1) concrete, we train a small CNN on the same data and augmentation, with **random initialisation** and the same callbacks. It is compared with the transfer-learning model on the *validation* set only (the test set stays untouched). `restore_best_weights=True` means the scores reported for it are those of its best-validation-loss epoch, not of its last epoch.

```
def build_scratch_cnn():
    m = keras.Sequential([keras.Input((IMG_SIZE, IMG_SIZE, 3))] + [
        l for f in (32, 64, 128) for l in (layers.Conv2D(f, 3, padding="same", activation="relu"), layers.BatchNormalization(),
layers.MaxPooling2D())
    ] + [layers.GlobalAveragePooling2D(), layers.Dropout(DROPOUT), layers.Dense(HEAD_UNITS, activation="relu"), layers.Dense(1,
activation="sigmoid")],
        name="scratch_cnn")
    compile_model(m, 1e-3)
    return m

scratch = build_scratch_cnn()
t0 = time.time()
hist_scratch = scratch.fit(train_ds, validation_data=val_ds, epochs=P1_EPOCHS, class_weight=class_weight, verbose=2,
                           callbacks=[keras.callbacks.EarlyStopping("val_loss", patience=ES_PATIENCE, restore_best_weights=True)])
scratch_val = scratch.evaluate(val_ds, verbose=0, return_dict=True)
print(f"\nScratch CNN: {scratch.count_params():,} parameters, trained {time.time() - t0:.0f}s | "
      f"val accuracy {scratch_val['accuracy']:.4f}, val recall {scratch_val['recall']:.4f}, val AUC {scratch_val['auc']:.4f}")
```

```
Epoch 1/12
200/200 - 61s - 307ms/step - accuracy: 0.6728 - auc: 0.7430 - loss: 0.5972 - precision: 0.6842 - recall: 0.6421 - val_accuracy: 0.5004 - val_auc:
0.6195 - val_loss: 0.7337 - val_precision: 0.0000e+00 - val_recall: 0.0000e+00
Epoch 2/12
200/200 - 53s - 264ms/step - accuracy: 0.8024 - auc: 0.8828 - loss: 0.4359 - precision: 0.8132 - recall: 0.7851 - val_accuracy: 0.5484 - val_auc:
0.5885 - val_loss: 0.6919 - val_precision: 0.5634 - val_recall: 0.4270
Epoch 3/12
200/200 - 56s - 279ms/step - accuracy: 0.8596 - auc: 0.9318 - loss: 0.3400 - precision: 0.8591 - recall: 0.8604 - val_accuracy: 0.6764 - val_auc:
0.7608 - val_loss: 0.6716 - val_precision: 0.6241 - val_recall: 0.8861
Epoch 4/12
200/200 - 57s - 286ms/step - accuracy: 0.8990 - auc: 0.9616 - loss: 0.2569 - precision: 0.8958 - recall: 0.9031 - val_accuracy: 0.7591 - val_auc:
0.8582 - val_loss: 0.5100 - val_precision: 0.7740 - val_recall: 0.7313
Epoch 5/12
200/200 - 65s - 324ms/step - accuracy: 0.9278 - auc: 0.9797 - loss: 0.1862 - precision: 0.9252 - recall: 0.9310 - val_accuracy: 0.8836 - val_auc:
0.9510 - val_loss: 0.3419 - val_precision: 0.9634 - val_recall: 0.7972
Epoch 6/12
200/200 - 70s - 348ms/step - accuracy: 0.9448 - auc: 0.9861 - loss: 0.1532 - precision: 0.9428 - recall: 0.9470 - val_accuracy: 0.8604 - val_auc:
0.9812 - val_loss: 0.2865 - val_precision: 0.9833 - val_recall: 0.7331
```

```
Epoch 7/12
200/200 - 69s - 345ms/step - accuracy: 0.9564 - auc: 0.9909 - loss: 0.1240 - precision: 0.9564 - recall: 0.9564 - val_accuracy: 0.9511 - val_auc:
0.9946 - val_loss: 0.1054 - val_precision: 0.9847 - val_recall: 0.9164
Epoch 8/12
200/200 - 70s - 350ms/step - accuracy: 0.9644 - auc: 0.9944 - loss: 0.0962 - precision: 0.9637 - recall: 0.9652 - val_accuracy: 0.9351 - val_auc:
0.9892 - val_loss: 0.1278 - val_precision: 0.9390 - val_recall: 0.9306
Epoch 9/12
200/200 - 69s - 346ms/step - accuracy: 0.9678 - auc: 0.9950 - loss: 0.0897 - precision: 0.9657 - recall: 0.9702 - val_accuracy: 0.8142 - val_auc:
0.9822 - val_loss: 0.4691 - val_precision: 0.9972 - val_recall: 0.6299
Epoch 10/12
200/200 - 68s - 341ms/step - accuracy: 0.9749 - auc: 0.9969 - loss: 0.0710 - precision: 0.9758 - recall: 0.9740 - val_accuracy: 0.9431 - val_auc:
0.9883 - val_loss: 0.1462 - val_precision: 0.9384 - val_recall: 0.9484
Epoch 11/12
200/200 - 76s - 378ms/step - accuracy: 0.9788 - auc: 0.9968 - loss: 0.0645 - precision: 0.9778 - recall: 0.9799 - val_accuracy: 0.9884 - val_auc:
0.9987 - val_loss: 0.0485 - val_precision: 0.9946 - val_recall: 0.9822
Epoch 12/12
200/200 - 76s - 382ms/step - accuracy: 0.9816 - auc: 0.9980 - loss: 0.0528 - precision: 0.9809 - recall: 0.9824 - val_accuracy: 0.9129 - val_auc:
0.9862 - val_loss: 0.2303 - val_precision: 0.9677 - val_recall: 0.8541
Scratch CNN: 110,785 parameters, trained 793s | val accuracy 0.9884, val recall 0.9822, val AUC 0.9987
```

7. Two-phase training

Phase 1 – train the head (base frozen), lr = 1e-3. The new dense layers start from random weights. Training them first, with the pretrained features fixed, gives a stable starting point.

Phase 2 – fine-tune (last 30 base layers unfrozen, BatchNorm kept frozen), lr = 1e-5. The top layers of the base hold the most task-specific features, so they are adapted to faces and drowsiness. The learning rate is 100x smaller, so the pretrained weights are nudged rather than overwritten. BatchNorm layers stay frozen because updating their statistics with small batches destabilises fine-tuning.

Callbacks (both phases). `EarlyStopping` (val loss, restore best weights), `ReduceLROnPlateau` (multiplies lr by 0.3 when val loss stalls) and `ModelCheckpoint` (keeps the best model by val loss in `outputs/best_model.keras`). In phase 2 the checkpoint only overwrites the file if it beats the best phase-1 val loss. Class weights are used only if the data is imbalanced (decided in section 5.1).

```python
BEST_PATH = OUT_DIR / "best_model.keras"

def callbacks(initial_best=None, min_lr=1e-6):
    return [keras.callbacks.EarlyStopping("val_loss", patience=ES_PATIENCE, restore_best_weights=True, verbose=1),
            keras.callbacks.ReduceLROnPlateau("val_loss", factor=LR_FACTOR, patience=LR_PATIENCE, min_lr=min_lr, verbose=1),
            keras.callbacks.ModelCheckpoint(BEST_PATH, monitor="val_loss", save_best_only=True,
                                            initial_value_threshold=initial_best, verbose=0)]

t_start = time.time()
print("=== Phase 1: frozen base, lr =", P1_LR)
h1 = model.fit(train_ds, validation_data=val_ds, epochs=P1_EPOCHS, class_weight=class_weight, callbacks=callbacks(), verbose=2)
t_phase1 = time.time() - t_start
print(f"Phase 1 took {t_phase1:.0f}s | best val_loss {min(h1.history['val_loss']):.4f}")
```

```
=== Phase 1: frozen base, lr = 0.001
Epoch 1/12
200/200 - 29s - 147ms/step - accuracy: 0.7984 - auc: 0.8809 - loss: 0.4332 - precision: 0.7989 - recall: 0.7977 - val_accuracy: 0.9484 - val_auc:
0.9905 - val_loss: 0.2229 - val_precision: 0.9390 - val_recall: 0.9591 - learning_rate: 0.0010
Epoch 2/12
200/200 - 25s - 126ms/step - accuracy: 0.9002 - auc: 0.9648 - loss: 0.2453 - precision: 0.9018 - recall: 0.8984 - val_accuracy: 0.9600 - val_auc:
0.9938 - val_loss: 0.1320 - val_precision: 0.9675 - val_recall: 0.9520 - learning_rate: 0.0010
Epoch 3/12
200/200 - 26s - 128ms/step - accuracy: 0.9385 - auc: 0.9838 - loss: 0.1667 - precision: 0.9430 - recall: 0.9335 - val_accuracy: 0.9760 - val_auc:
0.9991 - val_loss: 0.0830 - val_precision: 0.9908 - val_recall: 0.9609 - learning_rate: 0.0010
Epoch 4/12
200/200 - 26s - 130ms/step - accuracy: 0.9506 - auc: 0.9912 - loss: 0.1264 - precision: 0.9510 - recall: 0.9501 - val_accuracy: 0.9671 - val_auc:
0.9987 - val_loss: 0.0904 - val_precision: 0.9981 - val_recall: 0.9359 - learning_rate: 0.0010
Epoch 5/12
200/200 - 25s - 126ms/step - accuracy: 0.9595 - auc: 0.9937 - loss: 0.1059 - precision: 0.9595 - recall: 0.9595 - val_accuracy: 0.9902 - val_auc:
0.9997 - val_loss: 0.0392 - val_precision: 0.9859 - val_recall: 0.9947 - learning_rate: 0.0010
Epoch 6/12
200/200 - 26s - 128ms/step - accuracy: 0.9663 - auc: 0.9951 - loss: 0.0915 - precision: 0.9632 - recall: 0.9696 - val_accuracy: 0.9867 - val_auc:
0.9994 - val_loss: 0.0402 - val_precision: 0.9875 - val_recall: 0.9858 - learning_rate: 0.0010
Epoch 7/12
200/200 - 26s - 128ms/step - accuracy: 0.9708 - auc: 0.9966 - loss: 0.0763 - precision: 0.9691 - recall: 0.9727 - val_accuracy: 0.9876 - val_auc:
0.9995 - val_loss: 0.0358 - val_precision: 0.9841 - val_recall: 0.9911 - learning_rate: 0.0010
Epoch 8/12
200/200 - 26s - 128ms/step - accuracy: 0.9758 - auc: 0.9973 - loss: 0.0681 - precision: 0.9744 - recall: 0.9774 - val_accuracy: 0.9893 - val_auc:
0.9999 - val_loss: 0.0369 - val_precision: 1.0000 - val_recall: 0.9786 - learning_rate: 0.0010
Epoch 9/12
200/200 - 25s - 127ms/step - accuracy: 0.9813 - auc: 0.9982 - loss: 0.0567 - precision: 0.9806 - recall: 0.9821 - val_accuracy: 0.9938 - val_auc:
0.9999 - val_loss: 0.0275 - val_precision: 1.0000 - val_recall: 0.9875 - learning_rate: 0.0010
Epoch 10/12
200/200 - 28s - 138ms/step - accuracy: 0.9782 - auc: 0.9977 - loss: 0.0605 - precision: 0.9763 - recall: 0.9802 - val_accuracy: 0.9964 - val_auc:
0.9999 - val_loss: 0.0197 - val_precision: 0.9982 - val_recall: 0.9947 - learning_rate: 0.0010
Epoch 11/12
200/200 - 28s - 140ms/step - accuracy: 0.9788 - auc: 0.9983 - loss: 0.0542 - precision: 0.9799 - recall: 0.9777 - val_accuracy: 0.9956 - val_auc:
0.9998 - val_loss: 0.0187 - val_precision: 0.9964 - val_recall: 0.9947 - learning_rate: 0.0010
Epoch 12/12
200/200 - 28s - 140ms/step - accuracy: 0.9795 - auc: 0.9985 - loss: 0.0520 - precision: 0.9781 - recall: 0.9809 - val_accuracy: 0.9938 - val_auc:
0.9999 - val_loss: 0.0191 - val_precision: 0.9982 - val_recall: 0.9893 - learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 11.
Phase 1 took 317s | best val_loss 0.0187
```

```python
# Phase 2: unfreeze the last N layers of the base, but keep every BatchNorm layer frozen.
base.trainable = True
for layer in base.layers[:-UNFREEZE_LAST]:
    layer.trainable = False
```

```
for layer in base.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False
compile_model(model, P2_LR)                    # recompile so the new trainable flags take effect
n_train = sum(int(np.prod(w.shape)) for w in model.trainable_weights)
print(f"Phase 2: {n_train:,} trainable parameters (of {model.count_params():,}), lr = {P2_LR}")

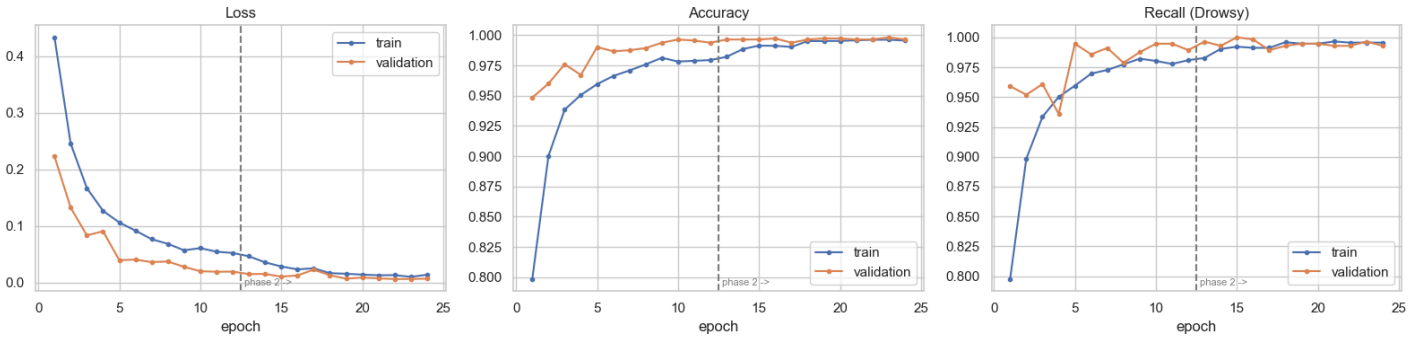
t0 = time.time()
h2 = model.fit(train_ds, validation_data=val_ds, epochs=P2_EPOCHS, class_weight=class_weight,
               callbacks=callbacks(initial_best=min(h1.history["val_loss"]), min_lr=1e-7), verbose=2)
t_phase2 = time.time() - t0
print(f"Phase 2 took {t_phase2:.0f}s | best val_loss {min(h2.history['val_loss']):.4f}")
```

```
Phase 2: 1,674,817 trainable parameters (of 2,422,081), lr = 1e-05
Epoch 1/12
200/200 - 41s - 203ms/step - accuracy: 0.9821 - auc: 0.9987 - loss: 0.0462 - precision: 0.9815 - recall: 0.9827 - val_accuracy: 0.9964 - val_auc:
0.9999 - val_loss: 0.0146 - val_precision: 0.9964 - val_recall: 0.9964 - learning_rate: 1.0000e-05
Epoch 2/12
200/200 - 32s - 158ms/step - accuracy: 0.9885 - auc: 0.9988 - loss: 0.0354 - precision: 0.9869 - recall: 0.9903 - val_accuracy: 0.9964 - val_auc:
0.9999 - val_loss: 0.0149 - val_precision: 1.0000 - val_recall: 0.9929 - learning_rate: 1.0000e-05
Epoch 3/12
200/200 - 31s - 157ms/step - accuracy: 0.9914 - auc: 0.9990 - loss: 0.0281 - precision: 0.9906 - recall: 0.9922 - val_accuracy: 0.9964 - val_auc:
1.0000 - val_loss: 0.0101 - val_precision: 0.9929 - val_recall: 1.0000 - learning_rate: 1.0000e-05
Epoch 4/12
200/200 - 31s - 156ms/step - accuracy: 0.9912 - auc: 0.9997 - loss: 0.0229 - precision: 0.9912 - recall: 0.9912 - val_accuracy: 0.9973 - val_auc:
0.9999 - val_loss: 0.0123 - val_precision: 0.9964 - val_recall: 0.9982 - learning_rate: 1.0000e-05
Epoch 5/12
Epoch 5: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.
200/200 - 31s - 154ms/step - accuracy: 0.9903 - auc: 0.9995 - loss: 0.0249 - precision: 0.9894 - recall: 0.9912 - val_accuracy: 0.9938 - val_auc:
0.9998 - val_loss: 0.0224 - val_precision: 0.9982 - val_recall: 0.9893 - learning_rate: 1.0000e-05
Epoch 6/12
200/200 - 31s - 156ms/step - accuracy: 0.9951 - auc: 0.9997 - loss: 0.0163 - precision: 0.9944 - recall: 0.9959 - val_accuracy: 0.9964 - val_auc:
0.9999 - val_loss: 0.0125 - val_precision: 1.0000 - val_recall: 0.9929 - learning_rate: 3.0000e-06
Epoch 7/12
200/200 - 31s - 154ms/step - accuracy: 0.9951 - auc: 0.9997 - loss: 0.0152 - precision: 0.9956 - recall: 0.9947 - val_accuracy: 0.9973 - val_auc:
1.0000 - val_loss: 0.0066 - val_precision: 1.0000 - val_recall: 0.9947 - learning_rate: 3.0000e-06
Epoch 8/12
200/200 - 31s - 155ms/step - accuracy: 0.9953 - auc: 0.9999 - loss: 0.0137 - precision: 0.9959 - recall: 0.9947 - val_accuracy: 0.9973 - val_auc:
1.0000 - val_loss: 0.0084 - val_precision: 1.0000 - val_recall: 0.9947 - learning_rate: 3.0000e-06
Epoch 9/12
Epoch 9: ReduceLROnPlateau reducing learning rate to 8.999999636216671e-07.
200/200 - 31s - 156ms/step - accuracy: 0.9958 - auc: 0.9999 - loss: 0.0125 - precision: 0.9950 - recall: 0.9965 - val_accuracy: 0.9964 - val_auc:
1.0000 - val_loss: 0.0070 - val_precision: 1.0000 - val_recall: 0.9929 - learning_rate: 3.0000e-06
Epoch 10/12
200/200 - 31s - 156ms/step - accuracy: 0.9962 - auc: 0.9997 - loss: 0.0128 - precision: 0.9969 - recall: 0.9956 - val_accuracy: 0.9964 - val_auc:
1.0000 - val_loss: 0.0059 - val_precision: 1.0000 - val_recall: 0.9929 - learning_rate: 9.0000e-07
Epoch 11/12
200/200 - 31s - 154ms/step - accuracy: 0.9962 - auc: 1.0000 - loss: 0.0097 - precision: 0.9969 - recall: 0.9956 - val_accuracy: 0.9982 - val_auc:
1.0000 - val_loss: 0.0060 - val_precision: 1.0000 - val_recall: 0.9964 - learning_rate: 9.0000e-07
Epoch 12/12
Epoch 12: ReduceLROnPlateau reducing learning rate to 2.6999999249710525e-07.
200/200 - 32s - 158ms/step - accuracy: 0.9956 - auc: 0.9996 - loss: 0.0134 - precision: 0.9956 - recall: 0.9956 - val_accuracy: 0.9964 - val_auc:
1.0000 - val_loss: 0.0065 - val_precision: 1.0000 - val_recall: 0.9929 - learning_rate: 9.0000e-07
Restoring model weights from the end of the best epoch: 10.
Phase 2 took 384s | best val_loss 0.0059
```

```
hist = pd.concat([pd.DataFrame(h1.history).assign(phase="1: head only"), pd.DataFrame(h2.history).assign(phase="2: fine-tune")], ignore_index=True)
hist.index.name = "epoch"; hist.index += 1
boundary = len(h1.history["loss"]) + 0.5

fig, axes = plt.subplots(1, 3, figsize=(16, 4))
for ax, (metric, title) in zip(axes, (("loss", "Loss"), ("accuracy", "Accuracy"), ("recall", "Recall (Drowsy)"))):
    ax.plot(hist.index, hist[metric], label="train", marker="o", ms=3)
    ax.plot(hist.index, hist[f"val_{metric}"], label="validation", marker="o", ms=3)
    ax.axvline(boundary, color="grey", ls="--"); ax.text(boundary + 0.2, ax.get_ylim()[0] + 0.02 * np.ptp(ax.get_ylim()), "phase 2 ->", color="grey",
fontsize=8)
    ax.set_title(title); ax.set_xlabel("epoch"); ax.legend()
save_show("training_curves")

best_val_loss = hist.val_loss.min()
best_phase = hist.loc[hist.val_loss.idxmin(), "phase"]
print(f"Best epoch by val_loss: {hist.val_loss.idxmin()} (phase {best_phase}) with val_loss {best_val_loss:.4f}")
display(hist.groupby("phase")[["loss", "val_loss", "accuracy", "val_accuracy", "val_recall"]].agg(["first", "last"]).round(4))
print(f"Total training time (both phases, incl. validation): {(t_phase1 + t_phase2) / 60:.1f} min")
```




```
Best epoch by val_loss: 22 (phase 2: fine-tune) with val_loss 0.0059
```

	loss		val_loss		accuracy		val_accuracy		val_recall	
	first	last	first	last	first	last	first	last	first	last
phase										
1: head only	0.4332	0.0520	0.2229	0.0191	0.7984	0.9795	0.9484	0.9938	0.9591	0.9893
2: fine-tune	0.0462	0.0134	0.0146	0.0065	0.9821	0.9956	0.9964	0.9964	0.9964	0.9929

```
Total training time (both phases, incl. validation): 11.7 min
```

Training results

- **Model size.** The model has 2,422,081 parameters in total: 2,257,984 in the frozen MobileNetV2 backbone (154 layers) and 164,097 trainable in the head during phase 1. In phase 2, 1,674,817 parameters are trainable (last 30 backbone layers with BatchNorm kept frozen, plus the head).
- **Phase 1 (lr 1e-3, 12 epochs, 317 s).** Validation accuracy was already 94.8% after the first epoch and 99.6% at the best epoch (epoch 11, val loss 0.0187).
- **Phase 2 (lr 1e-5, all 12 epochs run, 384 s).** Validation loss fell from 0.0187 to **0.0059** (the best epoch, epoch 22 overall, was saved as `best_model.keras`), with validation accuracy 99.64%. `ReduceLROnPlateau` lowered the learning rate three times in phase 2 (epochs 5, 9 and 12) and never in phase 1. The curves show a smooth continuation at the phase boundary and no sign of overfitting: training accuracy (99.6%) is about equal to validation accuracy (99.6%) although dropout and augmentation are active only in training.
- **Total training time:** 11.7 min for both phases on a 16-core CPU (no GPU).
- **Baseline from scratch (section 6.1).** A 110,785-parameter CNN reached a validation accuracy of **98.84%** (recall 98.22%) at its restored best epoch (epoch 11, lowest validation loss; the training callback restores those weights), while its last epoch scored 91.3%, so on this data a small scratch model also scores very high. It was far less stable: validation accuracy was 50.0% and 54.8% in the first two epochs (chance level), swung between 81.4% and 98.8% from epoch 6 on (81.4% at epoch 9, 91.3% at the last epoch), whereas the transfer model started at 94.8% and stayed at or above that throughout. Twelve scratch epochs also took 793 s, about 2.5x the 317 s of the twelve phase-1 epochs. Both models are judged on a *leaky* validation set, so the near-equal final accuracy says more about how easy frame-level classification is on this data than about the two methods.

8. Test-set evaluation

The best checkpoint is reloaded from disk (this is the model that is exported later) and evaluated **once** on the untouched test set at the default 0.5 threshold. Metrics, with *Drowsy as the positive class*:

- **Precision** = TP / (TP + FP): of the alerts raised, how many were real. Low precision = false alarms.
- **Recall** = TP / (TP + FN): of the drowsy drivers, how many we caught. Low recall = missed drowsy drivers.
- **F1** = harmonic mean of both. **Accuracy** = overall share correct.

```
best = keras.models.load_model(BEST_PATH, compile=False)   # inference only: no optimizer state needed
p_val = best.predict(val_ds, verbose=0).ravel()
p_test = best.predict(test_ds, verbose=0).ravel()


def metrics_at(y, p, thr):
    pred = (p >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y, pred, labels=[0, 1]).ravel()
    return dict(threshold=thr, accuracy=accuracy_score(y, pred), precision=precision_score(y, pred, zero_division=0),
                recall=recall_score(y, pred), f1=f1_score(y, pred), TP=tp, FN=fn, FP=fp, TN=tn)

m_test = metrics_at(y_test, p_test, 0.5)
print(classification_report(y_test, (p_test >= 0.5).astype(int), target_names=["Non-Drowsy (0)", "Drowsy (1)"], digits=4))
print("Majority-class baseline accuracy on test: %.4f" % max(y_test.mean(), 1 - y_test.mean()))
```

```
                precision    recall  f1-score   support

Non-Drowsy (0)     0.9905    0.9984    0.9944      1250
    Drowsy (1)     0.9984    0.9904    0.9944      1250

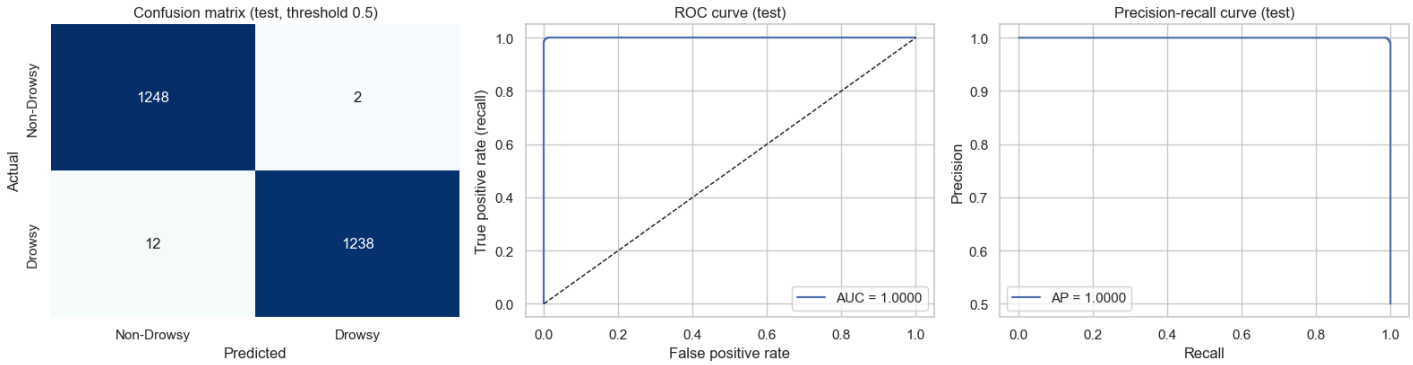
      accuracy                         0.9944      2500
     macro avg     0.9944    0.9944    0.9944      2500
  weighted avg     0.9944    0.9944    0.9944      2500

Majority-class baseline accuracy on test: 0.5000
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.3))
cm = confusion_matrix(y_test, (p_test >= 0.5).astype(int))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, ax=axes[0], xticklabels=["Non-Drowsy", "Drowsy"], yticklabels=["Non-Drowsy",
"Drowsy"])
axes[0].set_xlabel("Predicted"); axes[0].set_ylabel("Actual"); axes[0].set_title("Confusion matrix (test, threshold 0.5)")

fpr, tpr, _ = roc_curve(y_test, p_test); roc_auc = auc(fpr, tpr)
axes[1].plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}"); axes[1].plot([0, 1], [0, 1], "k--", lw=1)
axes[1].set_xlabel("False positive rate"); axes[1].set_ylabel("True positive rate (recall)"); axes[1].set_title("ROC curve (test)"); axes[1].legend()

prec, rec, _ = precision_recall_curve(y_test, p_test); ap = average_precision_score(y_test, p_test)
axes[2].plot(rec, prec, label=f"AP = {ap:.4f}"); axes[2].set_xlabel("Recall"); axes[2].set_ylabel("Precision")
axes[2].set_title("Precision-recall curve (test)"); axes[2].legend(loc="lower left")
save_show("test_evaluation")
print(f"ROC AUC {roc_auc:.4f} | Average precision {ap:.4f}")
```




```
ROC AUC 1.0000 | Average precision 1.0000
```

Test results at the default 0.5 threshold

Metric	Value
Accuracy	**99.44%**
Precision (Drowsy)	99.84%
Recall (Drowsy)	**99.04%**
F1	99.44%
Confusion matrix	TP 1238, FN 12, FP 2, TN 1248
ROC AUC / average precision	1.0000 / 1.0000 (rounded to four decimals)

- The majority-class baseline would score 50.0% on this balanced test set, so 99.44% is far above chance.
- **Which metric matters most: Drowsy recall.** A false negative means a drowsy driver is not warned, which is the event that causes crashes. A false positive costs an unnecessary beep. Precision still matters, because too many false alarms make drivers ignore or disable the system. Accuracy alone hides this trade-off.
- **Be careful with the near-perfect numbers.** An AUC of 1.0000 is not evidence of a perfect drowsiness detector. Section 5.2 showed that the test frames are near-duplicates of training frames, so this is mostly measuring recognition of already-seen people and sequences.

9. Confidence threshold analysis

The network outputs P(Drowsy). The default cut-off of 0.5 treats both errors as equally costly, which is wrong for a safety system. We sweep the threshold on the **validation set only** and choose the threshold that **maximises Drowsy recall among thresholds whose validation precision is at least 95%** (`MIN_PRECISION` , our definition of "acceptable precision"). If several thresholds tie on recall, the **highest** one is taken, because it raises the fewest false alarms. The test set is then evaluated at that single pre-chosen threshold, next to the default 0.5; it is never used to select the threshold.

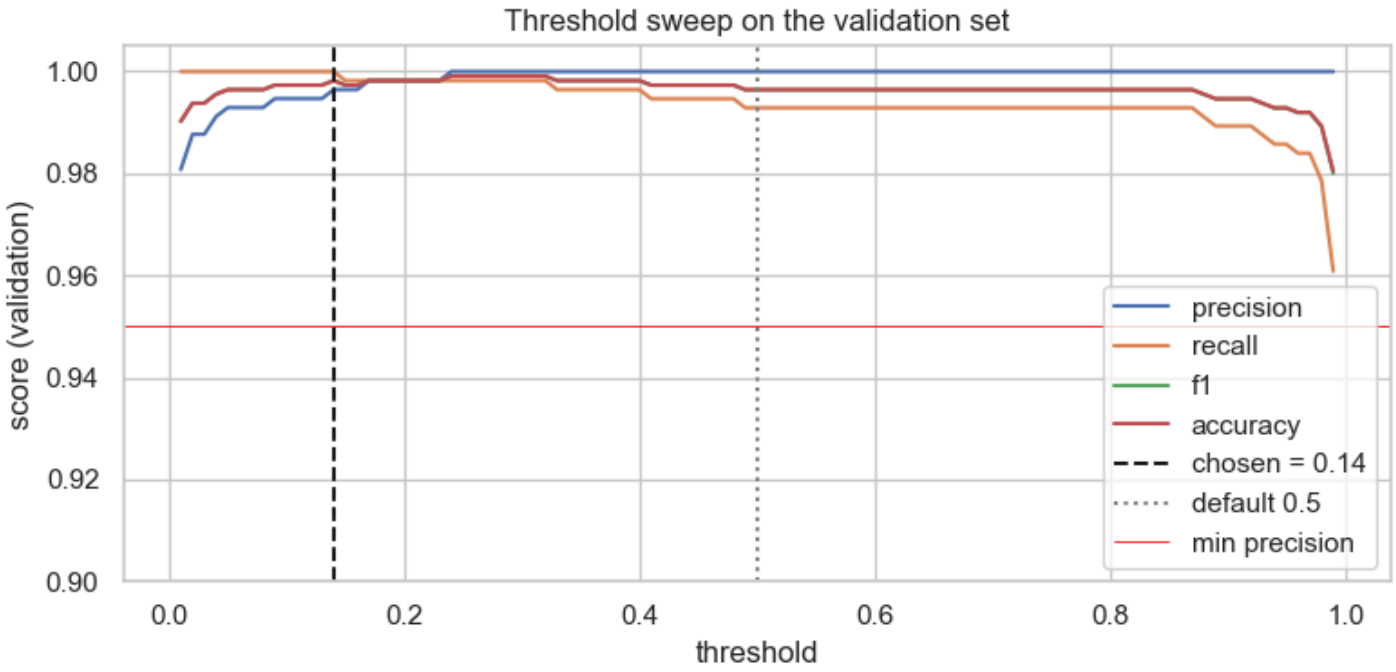
```
thresholds = np.round(np.arange(0.01, 1.00, 0.01), 2)
sweep = pd.DataFrame([metrics_at(y_val, p_val, t) for t in thresholds])
eligible = sweep[sweep.precision >= MIN_PRECISION]
THRESHOLD = float(eligible[eligible.recall == eligible.recall.max()].threshold.max())
print(f"Chosen threshold: {THRESHOLD} (max validation recall {eligible.recall.max():.4f} among thresholds with validation precision >=
{MIN_PRECISION:.0%}; ties -> highest)")

fig, ax = plt.subplots(figsize=(8, 4))
for col in ("precision", "recall", "f1", "accuracy"):
    ax.plot(sweep.threshold, sweep[col], label=col)
ax.axvline(THRESHOLD, color="k", ls="--", label=f"chosen = {THRESHOLD}"); ax.axvline(0.5, color="grey", ls=":", label="default 0.5")
ax.axhline(MIN_PRECISION, color="red", lw=0.6, label="min precision"); ax.set_ylim(0.9, 1.005); ax.set_xlabel("threshold"); ax.set_ylabel("score
(validation)"); ax.set_title("Threshold sweep on the validation set"); ax.legend()
save_show("threshold_sweep")

display(sweep[sweep.threshold.isin([0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9,
THRESHOLD])].drop_duplicates("threshold").set_index("threshold").round(4))

compare = pd.DataFrame({f"val @ {THRESHOLD}": metrics_at(y_val, p_val, THRESHOLD), "test @ 0.5 (default)": metrics_at(y_test, p_test, 0.5),
                        f"test @ {THRESHOLD} (chosen)": metrics_at(y_test, p_test, THRESHOLD)}).T
display(compare.round(4))
```

```
Chosen threshold: 0.14 (max validation recall 1.0000 among thresholds with validation precision >= 95%; ties -> highest)
```



threshold	accuracy	precision	recall	f1	TP	FN	FP	TN
0.10	0.9973	0.9947	1.0000	0.9973	562	0	3	560
0.14	0.9982	0.9965	1.0000	0.9982	562	0	2	561
0.20	0.9982	0.9982	0.9982	0.9982	561	1	1	562
0.30	0.9991	1.0000	0.9982	0.9991	561	1	0	563
0.40	0.9982	1.0000	0.9964	0.9982	560	2	0	563
0.50	0.9964	1.0000	0.9929	0.9964	558	4	0	563
0.60	0.9964	1.0000	0.9929	0.9964	558	4	0	563
0.70	0.9964	1.0000	0.9929	0.9964	558	4	0	563
0.80	0.9964	1.0000	0.9929	0.9964	558	4	0	563
0.90	0.9947	1.0000	0.9893	0.9946	556	6	0	563

	threshold	accuracy	precision	recall	f1	TP	FN	FP	TN
val @ 0.14	0.14	0.9982	0.9965	1.0000	0.9982	562.0	0.0	2.0	561.0
test @ 0.5 (default)	0.50	0.9944	0.9984	0.9904	0.9944	1238.0	12.0	2.0	1248.0
test @ 0.14 (chosen)	0.14	0.9952	0.9944	0.9960	0.9952	1245.0	5.0	7.0	1243.0

Choosing the threshold

- The validation sweep selected **0.14**: validation recall 1.0000 (562 of 562 drowsy found) with precision 0.9965 (2 false alarms). The curve is nearly flat between about 0.2 and 0.8 (validation accuracy 99.6% to 99.9%), because the validation set is saturated, so the choice is only weakly informed. On a subject-disjoint validation set the probabilities would be less extreme and the threshold would matter more.
- **Test, default vs chosen threshold:**

Threshold	Accuracy	Precision	Recall	F1	FN	FP
0.50 (default)	99.44%	99.84%	99.04%	99.44%	12	2
0.14 (chosen on validation)	**99.52%**	99.44%	**99.60%**	**99.52%**	**5**	7

- Lowering the threshold removed 7 of the 12 missed drowsy frames at the price of 5 additional false alarms among 1,250 non-drowsy frames. For a safety system that is the trade we want, although the net gain in accuracy is small (99.44% to 99.52%).
- Note that the first version of this analysis used the rule "highest threshold with validation recall >= 95%", which chose 0.99 and *reduced* test recall to 95.2%, because a recall floor says nothing about how far above the floor we are. The rule above (maximise recall subject to precision >= 95%) is the one reported.

10. Error analysis

We look at the test images the model gets wrong at the chosen threshold, ranked by confidence: the **most confidently wrong** images tell most about systematic failure. We then look for patterns in measurable properties (brightness, whether a face was found, source letter) to judge whether failures are *data problems* or *training problems*.

```
df_test["p_drowsy"] = p_test
df_test["pred"] = (p_test >= THRESHOLD).astype(int)
df_test["correct"] = df_test.pred == df_test.label
df_test["brightness"] = X_test.mean(axis=(1, 2, 3))
df_test["contrast"] = X_test.std(axis=(1, 2, 3))
```

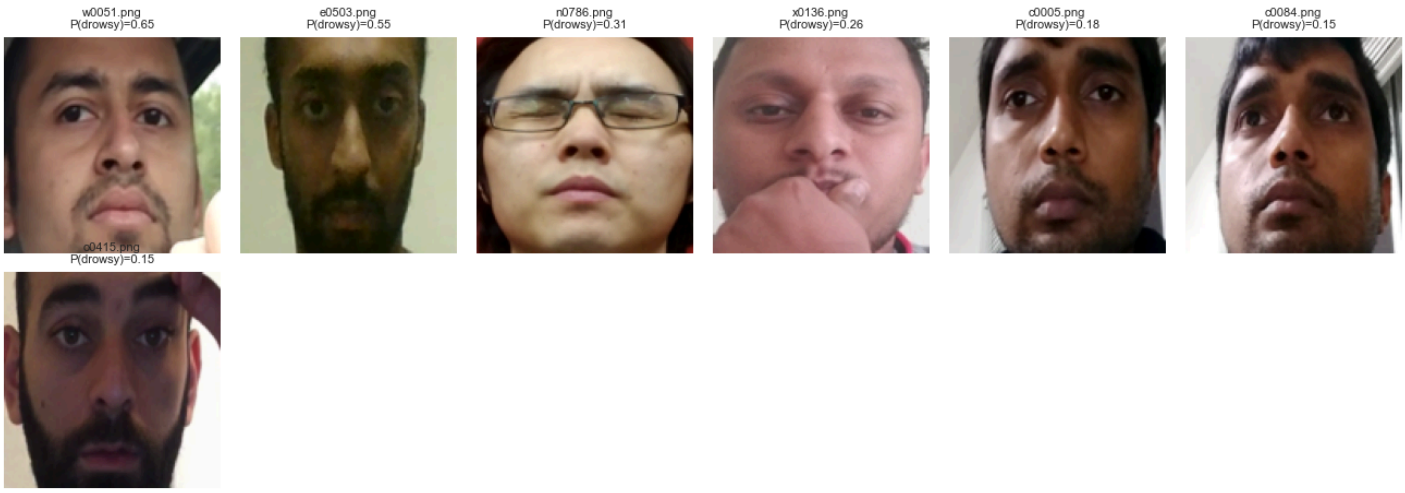
```
fn = df_test[(df_test.label == 1) & (df_test.pred == 0)].sort_values("p_drowsy")            # drowsy but predicted alert, most confident first
fp = df_test[(df_test.label == 0) & (df_test.pred == 1)].sort_values("p_drowsy", ascending=False)
print(f"At threshold {THRESHOLD}: {len(fn)} false negatives (missed drowsy), {len(fp)} false positives (false alarms)")

def show_errors(errors, title, fig_name, n=12):
    top = errors.head(n)
    if top.empty:
        print(f"{title}: none at threshold {THRESHOLD}")
        return
    show_images([X_test[i] for i in top.index], [f"{r.path.name}\nP(drowsy)={r.p_drowsy:.2f}" for r in top.itertuples()], ncols=6, cell=(2.2, 2.5),
suptitle=title)
    save_show(fig_name)

show_errors(fn, "False negatives: drowsy faces classified as alert (most confidently wrong first)", "false_negatives")
show_errors(fp, "False positives: alert faces classified as drowsy (most confidently wrong first)", "false_positives")
```

```
At threshold 0.14: 5 false negatives (missed drowsy), 7 false positives (false alarms)
```






```
# Measurable patterns: are the errors different from correct predictions?
summary = df_test.groupby(["class_name", "correct"]).agg(n=("path", "size"), brightness=("brightness", "mean"), contrast=("contrast", "mean"),
                                                          dlib_found=("dlib_found", "mean"), face_area=("face_area_frac", "mean")).round(3)
display(summary)

# Are errors concentrated in a few source sequences (letters)? That would point at data (subject/condition) rather than training.
per_seq = df_test.groupby("seq").agg(n=("path", "size"), errors=("correct", lambda s: int((~s).sum())))
per_seq["error_rate"] = per_seq.errors / per_seq.n
top_seq = per_seq.sort_values("errors", ascending=False).head(8)
display(top_seq.round(3))
print(f"Top-3 letters hold {top_seq.errors.head(3).sum()} of {per_seq.errors.sum()} errors ({top_seq.errors.head(3).sum() / max(per_seq.errors.sum(),
1):.0%}) "
      f"while holding {top_seq.n.head(3).sum() / per_seq.n.sum():.0%} of the test images")
```

class_name	correct	n	brightness	contrast	dlib_found	face_area
drowsy	**False**	5	106.473	56.224	1.000	0.805
	True	1245	99.429	53.119	0.933	0.736
non_drowsy	**False**	7	107.409	63.770	0.857	0.730
	True	1243	108.730	56.975	0.965	0.719

seq	n	errors	error_rate
N	74	5	0.068
c	31	2	0.065
n	57	1	0.018
e	65	1	0.015
x	80	1	0.012
w	35	1	0.029
o	37	1	0.027
d	54	0	0.000

```
Top-3 letters hold 8 of 12 errors (67%) while holding 6% of the test images
```

```python
# Stress test (evaluation only, nothing is tuned): how does the trained model cope with a darker cabin?
def evaluate_variant(transform, name):
    Xv = transform(X_test.astype("float32") / 255.0).clip(0, 1)
    p = best.predict(Xv, batch_size=128, verbose=0).ravel()
    return dict(condition=name, **{k: v for k, v in metrics_at(y_test, p, THRESHOLD).items() if k in ("accuracy", "precision", "recall", "f1")})

stress = pd.DataFrame([evaluate_variant(lambda x: x, "original"),
                       evaluate_variant(lambda x: x * 0.6, "darker (x0.6)"),
                       evaluate_variant(lambda x: x * 0.35, "dark (x0.35)"),
                       evaluate_variant(lambda x: x * 0.15, "very dark (x0.15)")]).set_index("condition")
display(stress.round(4))
dark_demo = [X_test[7], (X_test[7] * 0.35).astype("uint8"), (X_test[7] * 0.15).astype("uint8")]
show_images(dark_demo, ["original", "x0.35", "x0.15"], ncols=3, cell=(2.2, 2.4)); save_show("dark_stress_examples")
```

condition	accuracy	precision	recall	f1
original	0.9952	0.9944	0.9960	0.9952
darker (x0.6)	0.9776	0.9578	0.9992	0.9781
dark (x0.35)	0.8568	0.7826	0.9880	0.8734
very dark (x0.15)	0.5800	0.5437	0.9960	0.7034



What the errors look like (threshold 0.14: 5 false negatives, 7 false positives)

- **False negatives (5).** All five (N0346, N0355, N0474, N0350, N0318) appear to show the same driver (similar file-name prefix N and similar appearance: glasses, **head tilted back**), with the head tilted back so the chin and nose dominate and the eyelids are foreshortened. The model gives P(drowsy) between 0.03 and 0.11, so they are confident misses, not borderline cases.
- **False positives (7).** w0051 (0.65) is a face very close to the camera with an object cutting into the right edge of the frame. e0503 (0.55) is a dark, low-contrast face. n0786 (0.31) shows closed eyes behind glasses: either a blink or a questionable label. x0136 (0.26) has a hand over the mouth. c0005, c0084 and o0415 (0.15 to 0.18) are shadowed or dark faces with the head turned or tilted.
- **Prefix caveat.** The file-name prefix is not confirmed to be a subject ID (the documents do not say what it means), so "same driver" is an impression from the images and the shared prefix, not an established fact.
- **Patterns.** Failures cluster in specific conditions: **head pose** (tilted back), **glasses**, **low light or shadow** and **occlusion**. Prefix N alone accounts for 5 of the 12 errors (all false negatives) while holding 74 of the 2,500 test images (3%); the printed table of the three most error-prone prefixes holds 8 of 12 errors (67%) in 6% of the images.
- **Data problem or training problem?** The evidence points to the **data**. Training and validation curves show no overfitting or under-fitting, the global error rate is below 0.5%, and the remaining errors are concentrated in rare conditions and a few sequences that are under-represented, plus possible label noise at the boundary (a blink labelled non-drowsy). More of those conditions and cleaner labels would help more than extra epochs or a bigger model.
- **Darkness stress test** (evaluation only, nothing tuned). Dimming the test images makes the model flag alert drivers as drowsy while it keeps catching drowsy ones: at x0.6 accuracy is 97.8% with recall 99.9%, at x0.35 accuracy is 85.7% (precision falls to 78.3%, recall 98.8%) and at x0.15 accuracy is 58.0% (precision 54.4%, recall 99.6%). Recall stays high because almost everything dark is called drowsy, which is consistent with the model having partly learned "dark = drowsy" and matches the brightness gap noticed in section 3. In a real night-time cabin this would produce many false alarms, and at x0.15 the model is close to guessing "drowsy" for every image.

11. TensorFlow Lite conversion & edge deployment

The trained Keras model is exported and converted to four TFLite variants:

Variant	What changes	Typical use
float32	No optimisation, reference	Baseline
dynamic-range	Weights stored as int8, activations stay float and are quantized on the fly	Easy 4x size cut, no calibration data
float16	Weights stored as float16	Halves size, good on GPUs/NPUs with fp16
int8 (full integer)	Weights *and* activations int8, calibrated with 200 representative training images	Best for integer-only accelerators (Coral/EdgeTPU, many MCUs)

All variants are evaluated on the **full test set** with `tf.lite.Interpreter`, one image at a time (batch size 1, a single CPU thread to mimic an embedded device). Latency is the average wall-clock time per image on this PC's CPU. It is only a *relative* indicator, not a Raspberry Pi measurement.

```
SAVED_DIR = OUT_DIR / "saved_model"
with contextlib.redirect_stdout(io.StringIO()), contextlib.redirect_stderr(io.StringIO()):   # export logs thousands of tensor specs
    best.export(str(SAVED_DIR))
# The checkpoint written during training also stores Adam's optimizer state, which inflates its size.
# For a fair size comparison we save an inference-only .keras file (weights + architecture).
KERAS_INFERENCE_PATH = OUT_DIR / "drowsiness_model_inference.keras"
best.save(KERAS_INFERENCE_PATH)
print(f"Training checkpoint (with optimizer state): {BEST_PATH.stat().st_size / 1024:.0f} KB | inference-only .keras:
{KERAS_INFERENCE_PATH.stat().st_size / 1024:.0f} KB")

def representative_data():
    for x in X_train[rng.choice(len(X_train), 200, replace=False)]:
        yield [(x.astype("float32") / 255.0)[None]]

def convert(name, configure=lambda c: None):
    conv = tf.lite.TFLiteConverter.from_saved_model(str(SAVED_DIR))
    configure(conv)
    path = OUT_DIR / f"drowsiness_{name}.tflite"
    path.write_bytes(conv.convert())
    return path

def set_float16(c):
    c.optimizations = [tf.lite.Optimize.DEFAULT]; c.target_spec.supported_types = [tf.float16]

def set_int8(c):
    c.optimizations = [tf.lite.Optimize.DEFAULT]; c.representative_dataset = representative_data
    c.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]

tflite_paths = {
    "float32": convert("float32"),
    "dynamic_range": convert("dynamic_range", lambda c: setattr(c, "optimizations", [tf.lite.Optimize.DEFAULT])),
    "float16": convert("float16", set_float16),
    "int8": convert("int8", set_int8),
}
for name, p in tflite_paths.items():
    print(f"{name:14s} {p.stat().st_size / 1024:8.1f} KB  ->  {p}")
```

```
Training checkpoint (with optimizer state): 23096 KB | inference-only .keras: 9998 KB
float32          9301.9 KB  ->  outputs\drowsiness_float32.tflite
dynamic_range    2612.9 KB  ->  outputs\drowsiness_dynamic_range.tflite
float16          4683.5 KB  ->  outputs\drowsiness_float16.tflite
int8             2811.4 KB  ->  outputs\drowsiness_int8.tflite
```

```
def new_interpreter(path):
    interp = tf.lite.Interpreter(model_path=str(path), num_threads=1)
    interp.allocate_tensors()
    return interp, interp.get_input_details()[0], interp.get_output_details()[0]

def tflite_predict(path, X):
    """Run the interpreter image by image; returns probabilities and mean ms per image."""
    interp, inp, out = new_interpreter(path)
    probs = np.empty(len(X), dtype="float32")
    t0 = time.perf_counter()
    for i, x in enumerate(X):
        interp.set_tensor(inp["index"], (x.astype("float32") / 255.0)[None])
        interp.invoke()
        probs[i] = interp.get_tensor(out["index"])[0, 0]
    return probs, (time.perf_counter() - t0) / len(X) * 1000

interp, inp, out = new_interpreter(tflite_paths["int8"])
print("int8 model input :", inp["shape"], inp["dtype"].__name__, "| output:", out["shape"], out["dtype"].__name__)

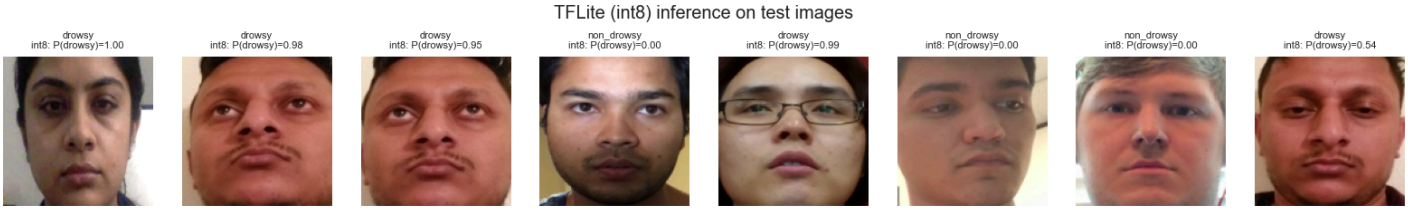
# Keras reference latency, same single-image protocol (graph-compiled, warmed up, default TF threading)
keras_fn = tf.function(lambda x: best(x, training=False))
keras_fn(X_test[:1].astype("float32") / 255.0)
t0 = time.perf_counter()
for x in X_test[:200]:
    keras_fn(x.astype("float32")[None] / 255.0)
keras_ms = (time.perf_counter() - t0) / 200 * 1000

rows, tflite_probs = [], {}
for name, path in tflite_paths.items():
    probs, ms = tflite_predict(path, X_test)
    tflite_probs[name] = probs
    m = metrics_at(y_test, probs, 0.5)
    rows.append(dict(model=name, size_KB=path.stat().st_size / 1024, accuracy=m["accuracy"], precision=m["precision"], recall=m["recall"],
f1=m["f1"],
                     recall_at_chosen_thr=metrics_at(y_test, probs, THRESHOLD)["recall"], ms_per_image=ms, max_abs_prob_diff_vs_keras=np.abs(probs -
p_test).max()))
rows.insert(0, dict(model="Keras (.keras)", size_KB=KERAS_INFERENCE_PATH.stat().st_size / 1024, accuracy=m_test["accuracy"],
precision=m_test["precision"], recall=m_test["recall"],
                    f1=m_test["f1"], recall_at_chosen_thr=metrics_at(y_test, p_test, THRESHOLD)["recall"], ms_per_image=keras_ms,
max_abs_prob_diff_vs_keras=0.0))
tflite_table = pd.DataFrame(rows).set_index("model")
tflite_table["size_vs_keras"] = (tflite_table.size_KB / tflite_table.size_KB.iloc[0]).round(3)
display(tflite_table.round(4))
tflite_table.to_csv(OUT_DIR / "tflite_comparison.csv")
```

```
int8 model input : [  1 128 128   3] float32 | output: [1 1] float32
```

model	size_KB	accuracy	precision	recall	f1	recall_at_chosen_thr	ms_per_image	max_abs_prob_diff_vs_keras	size_vs_keras
Keras (.keras)	9997.8252	0.9944	0.9984	0.9904	0.9944	0.9960	19.2571	0.0000	1.000
float32	9301.9336	0.9944	0.9984	0.9904	0.9944	0.9960	3.4498	0.0000	0.930
dynamic_range	2612.9375	0.9956	0.9984	0.9928	0.9956	0.9984	19.3093	0.4961	0.261
float16	4683.4570	0.9944	0.9984	0.9904	0.9944	0.9960	2.8514	0.0440	0.468
int8	2811.3672	0.9768	0.9745	0.9792	0.9769	0.9928	2.8769	0.9848	0.281

```
# Demonstration: the same 8 random test images through every TFLite variant (threshold 0.5, as in the table above)
demo = rng.choice(len(X_test), 8, replace=False)
demo_table = pd.DataFrame({"file": [df_test.path[i].name for i in demo], "true": [CLASS_NAMES[y_test[i]] for i in demo],
                           "keras": p_test[demo].round(3), **{n: tflite_probs[n][demo].round(3) for n in tflite_paths}})
show_images([X_test[i] for i in demo],
            [f"{CLASS_NAMES[y_test[i]]}\nint8: P(drowsy)={tflite_probs['int8'][i]:.2f}" for i in demo], ncols=8, cell=(2.0, 2.4),
            suptitle="TFLite (int8) inference on test images")
save_show("tflite_demo")
display(demo_table)
```



	file	true	keras	float32	dynamic_range	float16	int8
0	ZC0843.png	drowsy	1.000	1.000	1.000	1.00	0.996
1	X0734.png	drowsy	1.000	1.000	1.000	1.00	0.984
2	X0597.png	drowsy	1.000	1.000	1.000	1.00	0.949
3	s0327.png	non_drowsy	0.000	0.000	0.000	0.00	0.000
4	N0452.png	drowsy	0.157	0.157	0.412	0.16	0.988
5	i0325.png	non_drowsy	0.000	0.000	0.000	0.00	0.000
6	j0694.png	non_drowsy	0.000	0.000	0.000	0.00	0.000
7	X0309.png	drowsy	1.000	1.000	1.000	1.00	0.539

TFLite results (full test set, threshold 0.5, 1 CPU thread)

Model	Size	vs Keras	Accuracy	Recall	Latency / image
Keras `.keras` (inference only)	9,998 KB	1.00	99.44%	99.04%	19.3 ms (default threads)
TFLite float32	9,302 KB	0.93	99.44%	99.04%	3.4 ms
TFLite dynamic-range	**2,613 KB**	**0.26**	**99.56%**	99.28%	19.3 ms
TFLite float16	4,683 KB	0.47	99.44%	99.04%	2.9 ms
TFLite int8 (full integer)	2,811 KB	0.28	97.68%	97.92%	2.9 ms

(The training checkpoint `best_model.keras` is 23,096 KB because it also stores optimizer state; the table uses an inference-only `.keras` file for a fair comparison.)

- **float32** is numerically identical to Keras (max probability difference 0.0). **float16** is practically identical (max difference 0.04) at half the size and slightly faster.
- **Dynamic-range** is 3.8x smaller than the Keras model and did not lose accuracy, but it is about 5.6x *slower* than float32 in this single-thread run on this x86 PC (19.3 ms vs 3.4 ms), and individual probabilities can move by up to 0.50. Whether this slowdown exists on an ARM device such as a Raspberry Pi has to be measured there.
- **int8** is also about 3.6x smaller but lost **1.8 accuracy points** (97.68% vs 99.44%), and individual probabilities differ by up to 0.98 from the Keras model. With only 200 calibration images this version is not safe to ship for a safety-critical task without a larger calibration set, a check at the chosen threshold, or quantization-aware training. At the chosen threshold 0.14 its recall is 99.28% vs 99.60% for float32.
- **Latency caveat.** The numbers are wall-clock times per image on this PC and are only relative indicators. The 19.3 ms Keras figure uses a compiled graph with default threading, so it is not an equal-footing comparison with the single-thread TFLite rows.
- The `tf.lite.Interpreter` demo shows the same 8 test images through all variants. Float32 and float16 agree with the Keras model and dynamic-range stays on the same side of 0.5 for all eight, whereas the int8 scores differ visibly for some images (X0309: 1.00 float vs 0.54 int8; N0452: 0.16 float vs 0.99 int8). N0452 is the borderline frame: it is drowsy but Keras gives 0.157 and dynamic-range 0.412.

Trade-off. Float16 is the safest compact choice here (about half the size, same accuracy, same speed). Dynamic-range gives the smallest file with no accuracy loss but may be slow on some CPUs. Int8 is only worthwhile when the target accelerator requires integer-only arithmetic, and then it must be re-validated.

12. Ablation studies: face crop and two-phase training

Two short experiments check the design choices behind business questions Q4 and Q6 with measurements instead of arguments. Both use the **same seed, the same train/validation/test split indices and the same augmentation and callbacks** as the main run, and both are judged on the test set at the default 0.5 threshold (the test set is not used for any choice).

Experiment	What changes	Budget
A. No-crop	Same MobileNetV2 + head and the same two-phase recipe, but trained on the original images, only resized to 128x128 (no Dlib crop)	Same as the main run: 12 epochs + up to 12 epochs, early stopping active
B. Single-phase, whole base	The *entire* backbone is trainable from epoch 0, no frozen-head phase. Run at **lr 1e-3** (the phase-1 rate, aggressive) and at **lr 1e-5** (the fine-tuning rate)	One phase of up to 12 epochs each, early stopping active

Execution. These cells run in the same session as the rest of the notebook, directly after the main run, and reuse its splits, datasets, helper functions, recorded two-phase history `hist` and test predictions `p_test`. The main-run row in the comparison table therefore comes from the same execution. Ablation checkpoints go to a temporary folder and are not deliverables.

```
import tempfile
from sklearn.metrics import roc_auc_score

ABL_DIR = Path(tempfile.mkdtemp(prefix="ablation_"))   # checkpoints of the ablation runs (temporary)
ABL_SINGLE_EPOCHS = 12


def fit_logged(model, ckpt, train, val, epochs, lr, initial_best=None, min_lr=1e-6):
    """Compile and fit with the same callbacks as the main run; returns (history DataFrame, seconds)."""
    compile_model(model, lr)
    cbs = [keras.callbacks.EarlyStopping("val_loss", patience=ES_PATIENCE, restore_best_weights=True),
           keras.callbacks.ReduceLROnPlateau("val_loss", factor=LR_FACTOR, patience=LR_PATIENCE, min_lr=min_lr),
           keras.callbacks.ModelCheckpoint(ckpt, monitor="val_loss", save_best_only=True, save_weights_only=True,
                                           initial_value_threshold=initial_best)]
    t0 = time.time()
    h = model.fit(train, validation_data=val, epochs=epochs, class_weight=class_weight, callbacks=cbs, verbose=0)
    return pd.DataFrame(h.history), time.time() - t0


def train_two_phase(name, X_tr, y_tr, X_va, y_va):
    """Main-run recipe: phase 1 head only (P1_LR), phase 2 last UNFREEZE_LAST layers, BatchNorm frozen (P2_LR)."""
    tf.keras.utils.set_random_seed(SEED)
    train, val = make_dataset(X_tr, y_tr, training=True), make_dataset(X_va, y_va, training=False)
    m, base_model = build_model()
    ckpt = ABL_DIR / f"{name}.weights.h5"
    h1, t1 = fit_logged(m, ckpt, train, val, P1_EPOCHS, P1_LR)
    base_model.trainable = True
    for layer in base_model.layers[:-UNFREEZE_LAST]:
        layer.trainable = False
    for layer in base_model.layers:
        if isinstance(layer, layers.BatchNormalization):
            layer.trainable = False
    h2, t2 = fit_logged(m, ckpt, train, val, P2_EPOCHS, P2_LR, initial_best=h1.val_loss.min(), min_lr=1e-7)
    m.load_weights(ckpt)                       # best epoch over both phases, as in the main run
    h = pd.concat([h1.assign(phase="1"), h2.assign(phase="2")], ignore_index=True)
    return m, h, t1 + t2


def train_single_phase(name, lr, X_tr, y_tr, X_va, y_va):
    """Whole backbone trainable from epoch 0 (BatchNorm layers included, still called in inference mode)."""
    tf.keras.utils.set_random_seed(SEED)
    train, val = make_dataset(X_tr, y_tr, training=True), make_dataset(X_va, y_va, training=False)
    m, base_model = build_model()
    base_model.trainable = True
    ckpt = ABL_DIR / f"{name}.weights.h5"
    h, t = fit_logged(m, ckpt, train, val, ABL_SINGLE_EPOCHS, lr)
    m.load_weights(ckpt)
    return m, h, t, sum(int(np.prod(w.shape)) for w in m.trainable_weights)


def summarise(name, p, h, seconds):
    """One comparison-table row: test metrics at 0.5, AUC, validation loss, effort and disagreement with the main model."""
    r = metrics_at(y_test, p, 0.5)
    return {"model": name, "accuracy": r["accuracy"], "precision": r["precision"], "recall": r["recall"], "f1": r["f1"],
            "FN": r["FN"], "FP": r["FP"], "AUC": roc_auc_score(y_test, p), "best_val_loss": h.val_loss.min(),
            "epochs_run": len(h), "train_min": seconds / 60,
            "test_preds_differ_from_main": int(((p >= 0.5) != (p_test >= 0.5)).sum())}


rows = [summarise("Cropped, two-phase (main run)", p_test, hist, t_phase1 + t_phase2)]
curves = {"Two-phase (main run)": hist}
print("Ablation helpers ready | main-run test errors at 0.5: FN", m_test["FN"], "FP", m_test["FP"])
```

```
Ablation helpers ready | main-run test errors at 0.5: FN 12 FP 2
```

12.1 Experiment A: no face crop

The images are resized from 227x227 to 128x128 as they are, so hair, shoulders and background stay in the frame. The figure shows the same six faces cropped (top, the main pipeline) and uncropped (bottom).

```
X_nc = np.stack([cv2.resize(load_rgb(p), (IMG_SIZE, IMG_SIZE), interpolation=cv2.INTER_AREA) for p in df.path])   # same row order as X_all
demo_idx = tr_idx[:6]
show_images(list(X_all[demo_idx]) + list(X_nc[demo_idx]), ["Dlib crop"] * 6 + ["no crop"] * 6, ncols=6, cell=(2.0, 2.2),
            suptitle="Same faces: face-cropped input (top) vs original input (bottom)")
save_show("ablation_crop_vs_nocrop")

m_nc, h_nc, t_nc = train_two_phase("nocrop", X_nc[tr_idx], y_train, X_nc[va_idx], y_val)
p_nc = m_nc.predict(make_dataset(X_nc[te_idx], y_test, training=False), verbose=0).ravel()
rows.append(summarise("No crop, two-phase", p_nc, h_nc, t_nc))
print(f"No-crop run: {len(h_nc)} epochs in {t_nc / 60:.1f} min, best val_loss {h_nc.val_loss.min():.4f}")
```




```
No-crop run: 24 epochs in 15.4 min, best val_loss 0.0045
```

12.2 Experiment B: single-phase, whole base unfrozen from epoch 0

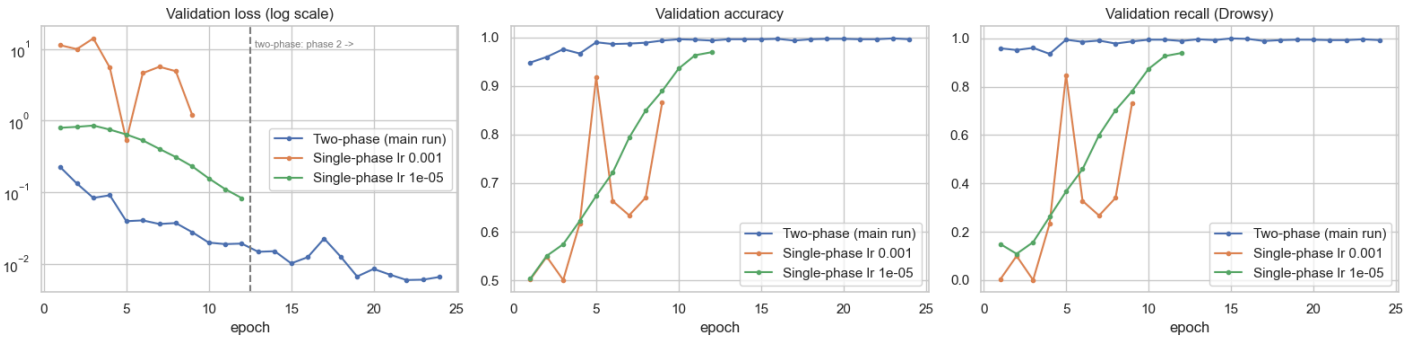
This is what "just fine-tune everything" looks like: the randomly initialised head and all 154 backbone layers receive gradients immediately. At lr 1e-3 the large early updates from the untrained head can flow into the pre-trained layers; at lr 1e-5 the update size is small but the head is learned slowly. BatchNorm layers are trainable here (their scale and shift), although the backbone is still called in inference mode.

```
for lr in (1e-3, 1e-5):
    m_sp, h_sp, t_sp, n_trainable = train_single_phase(f"single_{lr:g}", lr, X_train, y_train, X_val, y_val)
    p_sp = m_sp.predict(test_ds, verbose=0).ravel()
    label = f"Cropped, single-phase, whole base, lr {lr:g}"
    rows.append(summarise(label, p_sp, h_sp, t_sp))
    curves[f"Single-phase lr {lr:g}"] = h_sp
    print(f"{label}: {n_trainable:,} trainable parameters, {len(h_sp)} epochs in {t_sp / 60:.1f} min, best val_loss {h_sp.val_loss.min():.4f}")
```

```
Cropped, single-phase, whole base, lr 0.001: 2,387,969 trainable parameters, 9 epochs in 19.5 min, best val_loss 0.5308
Cropped, single-phase, whole base, lr 1e-05: 2,387,969 trainable parameters, 12 epochs in 27.0 min, best val_loss 0.0826
```

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))
for name, h in curves.items():
    x = np.arange(1, len(h) + 1)
    for ax, metric in zip(axes, ("val_loss", "val_accuracy", "val_recall")):
        ax.plot(x, h[metric], marker="o", ms=3, label=name)
for ax, title in zip(axes, ("Validation loss (log scale)", "Validation accuracy", "Validation recall (Drowsy)")):
    ax.set_title(title); ax.set_xlabel("epoch"); ax.legend()
axes[0].set_yscale("log")
boundary = (hist.phase == "1: head only").sum() + 0.5
axes[0].axvline(boundary, color="grey", ls="--"); axes[0].text(boundary + 0.3, axes[0].get_ylim()[1] * 0.5, "two-phase: phase 2 ->", color="grey",
fontsize=8)
save_show("ablation_single_vs_two_phase_curves")

comparison = pd.DataFrame(rows).set_index("model")
display(comparison.round(4))
comparison.to_csv(OUT_DIR / "ablation_comparison.csv")
```



model	accuracy	precision	recall	f1	FN	FP	AUC	best_val_loss	epochs_run	train_min	test_preds_differ_from_main
Cropped, two-phase (main run)	0.9944	0.9984	0.9904	0.9944	12	2	1.0000	0.0059	24	11.6845	0
No crop, two-phase	0.9984	0.9976	0.9992	0.9984	1	3	1.0000	0.0045	24	15.3757	16
Cropped, single-phase, whole base, lr 0.001	0.8940	0.9824	0.8024	0.8833	247	18	0.9829	0.5308	9	19.4736	263
Cropped, single-phase, whole base, lr 1e-05	0.9728	0.9966	0.9488	0.9721	64	4	0.9986	0.0826	12	27.0119	74

What the ablations show (test set, threshold 0.5)

Model	Accuracy	Precision	Recall	F1	FN / FP	AUC	Best val loss	Epochs	Train time
Cropped, two-phase (main run)	99.44%	99.84%	99.04%	99.44%	12 / 2	1.0000	0.0059	24	11.7 min
A. No crop, two-phase	99.84%	99.76%	99.92%	99.84%	1 / 3	1.0000	0.0045	24	15.4 min
B1. Single-phase, whole base, lr 1e-3	89.40%	98.24%	80.24%	88.33%	247 / 18	0.9829	0.5308	9	19.5 min
B2. Single-phase, whole base, lr 1e-5	97.28%	99.66%	94.88%	97.21%	64 / 4	0.9986	0.0826	12	27.0 min

Experiment A, no crop: no benefit from the Dlib crop on this dataset. The uncropped model scored the same or slightly better (accuracy 99.84% vs 99.44%, recall 99.92% vs 99.04%, validation loss 0.0045 vs 0.0059) with the same number of epochs. The two models disagree on 16 of the 2,500 test images (the printed column `test_preds_differ_from_main`; per-image predictions were not saved). With a binary label, an image that both models get wrong receives the same prediction from both, so every one of the 16 disagreements is an image that exactly one model gets wrong. Together with the error totals (14 vs 4) this implies 13 images wrong only for the cropped model and 3 only for the no-crop model (and 1 wrong for both). The exact two-sided McNemar p-value for 13 vs 3 is about 0.02 (it would lie between about 0.002 and 0.03 for any split of the errors that the totals allow). The no-crop run took longer (15.4 vs 11.7 min) with the same epoch count; wall-clock times on this shared CPU vary and the difference is not something to interpret. With one seed per model and a leaky test set, read this as "no measurable advantage for cropping", not as proof that cropping hurts. The figure explains why: the supplied images are already tight face crops, so the Dlib step only trims a border. The Dlib pass also costs about 320 s of preprocessing for 10,000 images, which is not included in the training times above.

Experiment B, single-phase: the two-phase recipe is clearly better.

- **lr 1e-3, whole base from epoch 0 (B1).** The curves show the textbook failure. Validation loss started at roughly 10 to 14 (read from the plot) with validation accuracy at about 50% (chance) for the first epochs, because the large updates driven by the untrained head damaged the pre-trained features. It reached its best validation loss (0.5308, validation accuracy about 92%) at epoch 5, then became unstable again (validation accuracy about 63% to 69% in epochs 6 to 8) and early stopping ended the run after 9 epochs. The restored epoch-5 model is **90x worse** in validation loss than the two-phase 0.0059, took 19.5 min, and on the test set it missed **247** drowsy frames against 12 (recall 80.24% vs 99.04%) with 18 false alarms.
- **lr 1e-5, whole base from epoch 0 (B2).** Stable but far too slow: the randomly initialised head learns slowly at this rate, and after the 12-epoch cap validation loss was still falling (0.0826), test recall was only 94.88% (**64** missed drowsy frames) and AUC 0.9986. It took 27.0 min, 2.3x the two-phase run.
- **Caveats.** B2 used the full 12-epoch cap (the two-phase run used 24 epochs, 12 + 12) and B1 was stopped early after 9, so longer runs would close part of the gap, especially for B2. One seed per run, and the aggressive-lr run is erratic: an earlier execution of this notebook gave the same B1 recipe a recall of 98.32% (21 missed), so the size of its failure is not stable, only its direction. The comparison shows that the single-phase variants are slower and less reliable at reaching the same quality, not that they can never match it.

13. Business questions

The answers below are labelled Q1 to Q8 in the order of the problem statement, plus the bonus Q9 for the fleet manager. Before answering, one more experiment supports the fleet-manager question (Q9, bonus): **temporal smoothing**. In a real vehicle the model sees a *stream* of frames, and acting on one frame is risky. The test frames are grouped by source letter and ordered by frame number to form pseudo-sequences (frames are *not* consecutive video frames, so this is only an approximation of a real video). A rolling mean of P(Drowsy) over `SMOOTH_WINDOW` frames is compared with the threshold.

```
def alert_rates(frame_df, thr, window):
    """Per-class share of frames flagged raw vs. smoothed, and share of sequences with at least one alert."""
    out = []
    for label, g_label in frame_df.groupby("label"):
        raw_frames = smooth_windows = raw_seq = smooth_seq = n_seq = 0
        n_frames = n_windows = 0
        for _, g in g_label.groupby("seq"):
            if len(g) < window:
                continue
            p = g.sort_values("frame").p_drowsy.to_numpy()
            roll = np.convolve(p, np.ones(window) / window, mode="valid")
            raw, smooth = p >= thr, roll >= thr
            n_frames += len(p); n_windows += len(roll); raw_frames += raw.sum(); smooth_windows += smooth.sum()
            n_seq += 1; raw_seq += raw.any(); smooth_seq += smooth.any()
        if n_seq == 0:
            continue
        out.append({"class": CLASS_NAMES[label], "sequences": n_seq, "single-frame alert rate": raw_frames / n_frames,
                    f"{window}-frame smoothed alert rate": smooth_windows / n_windows,
                    "sequences with >=1 raw alert": raw_seq / n_seq, "sequences with >=1 smoothed alert": smooth_seq / n_seq})
    return pd.DataFrame(out).set_index("class")

display(alert_rates(df_test, THRESHOLD, SMOOTH_WINDOW).round(4))
print("Note: for non_drowsy rows an alert is a FALSE alarm; for drowsy rows it is a DETECTION.")
```

class	sequences	single-frame alert rate	10-frame smoothed alert rate	sequences with >=1 raw alert	sequences with >=1 smoothed alert
non_drowsy	25	0.0056	0.0	0.24	0.0
drowsy	27	0.9960	1.0	1.00	1.0

```
Note: for non_drowsy rows an alert is a FALSE alarm; for drowsy rows it is a DETECTION.
```

Q1. Why not start from scratch?

Transfer learning reuses what a network pre-trained on ImageNet (about 1.2 M images) has already learned (edges, textures, parts), so only a small head needs to be learned from our 6,375 training images. In this notebook the transfer model reached **94.8% validation accuracy after one epoch**, and its curves were smooth, whereas the scratch CNN was at chance level (50.0%, 54.8%) for its first two epochs, had an unstable validation curve (81.4% at epoch 9, 91.3% at epoch 12) and needed 793 s for 12 epochs, about 2.5x the 317 s of the 12 phase-1 epochs. In fairness, the scratch model reached 98.84% validation accuracy at its restored best epoch (epoch 11; its last epoch was 91.3%) on this easy, leaky validation set, so the evidence for transfer learning here is faster, more stable training and less compute, not a large accuracy gap. With less data, harder conditions (night, other drivers) or limited compute the pre-trained features matter more.

Q2. The cost of missing a drowsy driver

A false negative (drowsy predicted as alert) leaves a fatigued driver unwarned, which can mean a crash. A false positive (alert predicted as drowsy) causes an unnecessary beep, an inconvenience. With the default 0.5 threshold the test set contains **12 missed drowsy frames and 2 false alarms**. Lowering the threshold to 0.14 (chosen on validation) gives **5 missed and 7 false alarms**: recall rises from 99.04% to 99.60% while precision falls from 99.84% to 99.44%. We therefore optimise **Drowsy recall** first, with precision as a constraint, because too many false alarms eventually lead drivers to ignore the system.

Q3. Seeing in the dark

Our augmentation changes brightness by only about +/-25%, so the model has effectively seen only normally lit faces. The stress test shows what that means: at x0.6 brightness it still works (accuracy 97.8%, recall 99.9%), but at x0.35 accuracy drops to 85.7% and at x0.15 to 58.0%, with precision falling to 54.4%. The dominant failure is **false alarms**, consistent with the model associating dark images with drowsiness (drowsy training images are darker on average: 100.1 vs 109.9). Recall stays high (98.8% and 99.6%) because nearly every dark image is called drowsy, so the model is not detecting anything useful at x0.15. A deployable system needs night-time and infrared (IR) camera data, stronger brightness/contrast/gamma augmentation and a lighting check.

Q4. Why detect the face first?

In principle the camera sees much more than the face (hair, shoulders, headrest, window, background), and isolating the face spends the 128x128 input on the eyes, mouth and head pose and removes background shortcuts. We tested this claim directly. Dlib found a face in **95.0%** of the images (Haar 91.5%, at least one 98.2%) and the Dlib box covers a median **63%** of the image, so the crop does remove a band of non-face pixels. The **no-crop ablation (section 12)** trained the same model with the same recipe on the original images, only resized: it scored **99.84% test accuracy and 99.92% recall (1 FN, 3 FP)** against **99.44% and 99.04% (12 FN, 2 FP)** for the cropped pipeline, with validation loss 0.0045 vs 0.0059. So on this dataset the face crop gave **no measurable benefit**; the uncropped model was equal or slightly better (the models disagree on 16 test images, which with the error totals implies 13 wrong only for the cropped model and 3 only for the no-crop model; McNemar p of about 0.02; one seed and a leaky test set, so weak evidence at most). The likely reasons are that the supplied images are already tight Viola-Jones face crops, so Dlib only trims a border, and that the test frames share people and backgrounds with the training frames (section 5.2), so background shortcuts do not hurt on this test. The Dlib step also costs about 320 s of preprocessing for 10,000 images and falls back to the original image for ~5%. The argument for face detection therefore rests on deployment rather than on these results: a real in-cabin frame is not pre-cropped and contains far more background, and the detector also gives a "driver absent / camera blocked" signal. That benefit would have to be tested on uncropped in-cabin footage, which this dataset does not provide.

Q5. Shrinking the model for the road

The inference-only Keras model is **9,998 KB**; TFLite float32 is 9,302 KB, **float16 4,683 KB (0.47x)**, **dynamic-range 2,613 KB (0.26x, 3.8x smaller)** and **int8 2,811 KB (0.28x)**. An in-vehicle camera has a small CPU, little memory and no connection to a cloud GPU, and needs a response within a fraction of a second, so on-device size and latency matter. Accuracy was unchanged for float16 (99.44%) and dynamic-range (99.56%), but int8 dropped to 97.68%. On this PC float32 took 3.4 ms per image on one thread, float16 and int8 2.9 ms, and dynamic-range 19.3 ms, so the smallest file is not automatically the fastest. Recommendation: float16, after measuring latency on the real target device.

Q6. Training in two phases

Phase 1 trains only the new, randomly initialised head at lr **1e-3** on top of frozen features. Phase 2 unfreezes the last 30 backbone layers (BatchNorm kept frozen) at **1e-5**, a hundred times smaller. This improved validation loss from 0.0187 to **0.0059**, and the curves continue smoothly across the boundary. The **single-phase ablation (section 12)** measured what happens if everything is updated from the start. With the whole backbone trainable at lr 1e-3, validation accuracy sat near chance (about 50%) in the first epochs and validation loss started at roughly 10 to 14, because the large, noisy gradients of the untrained head overwrote useful ImageNet features. Its best validation loss (epoch 5) was **0.5308, about 90x worse than 0.0059**, it became unstable again and was stopped after 9 epochs (19.5 min), and it missed **247** drowsy test frames instead of 12 (recall 80.24% vs 99.04%). With the whole backbone trainable at the small fine-tuning rate 1e-5, training was stable but too slow: after 12 epochs (27.0 min vs 11.7 min for the two-phase run) validation loss was still 0.0826 and test recall only **94.88% (64 missed drowsy frames)**. So the two-phase recipe is both faster and more reliable here. These runs were capped at 12 epochs with one seed (the two-phase run used 24 epochs), and the lr 1e-3 outcome varied a lot between executions of this notebook, so longer or repeated single-phase runs could narrow the gap.

Q7. Beyond just accuracy

A model that always predicts the majority class would score high accuracy on imbalanced data and would never catch a drowsy driver. On this exactly balanced test set such a model would get 50.0%, so accuracy is at least meaningful here, but the full report is still more informative: precision 99.84% and recall 99.04% at 0.5 reveal that 12 drowsy frames were missed against 2 false alarms, and the threshold analysis shows the balance can be shifted deliberately. In real fleet data drowsy frames are rare, so accuracy would be dominated by the alert class and could look excellent for a model with poor recall. We would not trust an accuracy figure on its own.

Q8. Improving the model

The confusion matrix shows very few errors (5 FN, 7 FP at threshold 0.14), and the training curves show no overfitting, so the remaining failures look like **data problems** rather than training problems: head tilted back with glasses (all five false negatives, N0346, N0355, N0474, N0350, N0318, which appear to show one driver), dark or shadowed faces (e0503, c0005, c0084, o0415), an object cutting into the frame edge (w0051), a hand over the mouth (x0136) and a possibly mislabelled blink (n0786). Two practical steps come first:

1. **Collect more diverse data for the observed failure conditions:** more drowsy and alert examples with the head tilted back, with glasses, and in dark or shadowed light, including night/IR footage.
2. **Use a subject-disjoint split and relabel the boundary cases:** obtain subject IDs (the file-name prefix is not confirmed to be one) so that train, validation and test contain different drivers, because the current split is leaky (section 5.2); and have the boundary frames reviewed and relabelled (for example n0786, a closed-eyes frame labelled non-drowsy).

Further options: stronger brightness and pose augmentation; temporal context (sequences of frames, eye-closure duration) instead of single frames; re-calibrating or retraining the int8 model with more calibration data. More epochs or a bigger backbone are unlikely to help.

Q9 (bonus): Actionable advice (the fleet manager's view)

A prediction is only useful if it triggers the right action. The pipeline from detection to action:

1. **Camera (IR-capable, facing the driver)** -> frame every few hundred milliseconds.
2. **Face detection (Dlib/OpenCV)** -> crop. If *no face* is found for several seconds, raise a separate "camera blocked / driver absent" event instead of guessing.
3. **TFLite classifier on the device** -> P(drowsy) per frame, with the threshold chosen for high recall (0.14 here).
4. **Smoothing over a window of frames before alerting.** Single frames are noisy (blinks, glare). In our pseudo-sequence test (test frames grouped by prefix and ordered by frame number, which are not consecutive video frames, so only an approximation) a 10-frame rolling mean cut single-frame false alarms from 0.56% of non-drowsy frames to 0.0%. Without smoothing **24% (6 of 25)** of the non-drowsy sequences produced at least one false alert; with smoothing **none** did, while **100%** (27 of 27) of drowsy sequences were still detected.
5. **Escalating response:** first an in-cab audio or seat-vibration alert; if drowsiness persists or recurs, a message to the driver to stop at the next safe place and a notification to the dispatcher or fleet-management portal.
6. **Fleet-level use:** log events with time, route and shift length to find risky shifts, routes and fatigue-prone times of day, and adjust schedules and rest-stop planning accordingly.
7. **Governance:** tune the threshold on the fleet's own data, track missed events and false alarms as KPIs, protect driver privacy (process on the device, store events and not video) and keep a human in the loop for any employment decision.

The main caution is that these numbers come from a leaky frame-level test: validate on drivers, vehicles and lighting the model has never seen before relying on it.

14. Conclusion

```
final = pd.DataFrame({
    "Keras @ 0.5": metrics_at(y_test, p_test, 0.5),
    f"Keras @ {THRESHOLD} (chosen on val)": metrics_at(y_test, p_test, THRESHOLD),
    "TFLite int8 @ 0.5": metrics_at(y_test, tflite_probs["int8"], 0.5),
}).T[["accuracy", "precision", "recall", "f1", "TP", "FN", "FP", "TN"]]
display(final.round(4))
final.to_csv(OUT_DIR / "final_results.csv")
print("Saved artefacts:"); [print(" ", p) for p in sorted(OUT_DIR.glob("*")) if p.is_file()]
```

	accuracy	precision	recall	f1	TP	FN	FP	TN
Keras @ 0.5	0.9944	0.9984	0.9904	0.9944	1238.0	12.0	2.0	1248.0
Keras @ 0.14 (chosen on val)	0.9952	0.9944	0.9960	0.9952	1245.0	5.0	7.0	1243.0
TFLite int8 @ 0.5	0.9768	0.9745	0.9792	0.9769	1224.0	26.0	32.0	1218.0

```
Saved artefacts:
  outputs\ablation_comparison.csv
  outputs\best_model.keras
  outputs\drowsiness_dynamic_range.tflite
  outputs\drowsiness_float16.tflite
  outputs\drowsiness_float32.tflite
  outputs\drowsiness_int8.tflite
  outputs\drowsiness_model_inference.keras
  outputs\final_results.csv
  outputs\tflite_comparison.csv
```

Results summary

Item	Result
Data	7,500 train (6,375 train + 1,125 val after the split), 2,500 test, perfectly balanced; 0 corrupted files, 0 duplicates
Face detection	Dlib 95.0%, Haar 91.5%, at least one 98.2% (fallback to the original image otherwise)
Model	MobileNetV2 (ImageNet) + head; 2.42 M parameters; two-phase training, 11.7 min on CPU
Test @ 0.5	Accuracy 99.44%, precision 99.84%, recall 99.04%, F1 99.44% (FN 12, FP 2)
Test @ 0.14 (chosen on validation)	Accuracy 99.52%, precision 99.44%, **recall 99.60%**, F1 99.52% (FN 5, FP 7)
Brightness stress test (x0.35 / x0.15)	Accuracy 85.7% / 58.0% (false alarms rise; recall 98.8% / 99.6%)
TFLite	float16 4,683 KB (accuracy 99.44%); dynamic-range 2,613 KB (99.56%); int8 2,811 KB (97.68%); Keras inference model 9,998 KB
Ablation A: no face crop	Test accuracy 99.84%, recall 99.92% (1 FN, 3 FP) vs 99.44% / 99.04% (12 FN, 2 FP) with the crop: no benefit from cropping on this data
Ablation B: single-phase, whole base	lr 1e-3: recall 80.24% (247 FN), val loss 0.5308, 9 epochs, 19.5 min; lr 1e-5: recall 94.88% (64 FN), val loss 0.0826, 27.0 min; two-phase: recall 99.04% (12 FN), val loss 0.0059, 11.7 min
Smoothing (10 frames)	Non-drowsy sequences with a false alert 24% -> 0%; drowsy sequences detected 100% -> 100%

Limitations

- **Frame-level only.** Each image is classified on its own. Real drowsiness is a *temporal* state (eye-closure duration, blink rate, nodding), which a single frame cannot capture. The smoothing experiment is only a rough proxy, as the test frames are not consecutive video frames.
- **Possible subject and near-duplicate leakage.** There is no subject ID. Validation and test images have a near-identical training image (median pixel distance 0.86-0.87 vs 8.68 for a random one; 78% within 5 frame numbers of a training frame with the same prefix; 54 of 54 test prefixes also appear in train). The near-perfect scores (AUC 1.0000) overstate how well the model will do on unseen drivers, and threshold selection on a saturated validation set is weakly informed.
- **Frontal-face bias.** The data are mostly frontal faces and Dlib's HOG detector is frontal-only; detection is lower for drowsy faces (about 93%), the typical failures are tilted heads and glasses, and we have no profile views, masks, sunglasses or night/IR images.
- **Acquisition shortcut.** Drowsy images are darker on average and the model reacts to darkening with false alarms.
- **Label noise and short ablations.** Blinks may be labelled non-drowsy. The two ablations in section 12 are single-seed and short (the single-phase runs are capped at 12 epochs), and the face-crop comparison is made on a leaky test set, so they show direction rather than precise effect sizes. Latency was measured on a PC, not on the target device.
- **Run-to-run variation.** Results come from one seed per run and CPU training is not bit-reproducible. Earlier executions of this notebook gave different numbers than this final single-session run (for example main test accuracy 99.64% instead of 99.44% at 0.5, chosen threshold 0.18 instead of 0.14, and recall 98.32% instead of 80.24% for the single-phase lr 1e-3 ablation), so differences of a few test images, and the exact error images listed in section 10, should not be over-interpreted.

Next steps

1. Obtain subject IDs (or split by source video) and re-evaluate with a subject-disjoint test set.
2. Add night/IR, glasses, sunglasses, head-pose and occlusion data; strengthen brightness and pose augmentation.
3. Move from frames to short clips: eye-closure ratio or PERCLOS, blink rate and head-pose features, or a small temporal model on top of the CNN embeddings.
4. Re-calibrate or use quantization-aware training for int8, and benchmark all variants on the real target hardware (for example Raspberry Pi or an Edge TPU).
5. Pilot with a small fleet, track missed events and false alarms, and tune the threshold and smoothing window on that data.